



3.0

DOCUMENTO DE DISEÑO DEL SISTEMA

Plataforma de Gestión Integral de NNA

Proyecto	Xolix — Fundación de Restitución de Derechos de NNA
Version	3.0
Fecha	Junio 2026
Institucion	Escuela Superior de Computo — IPN
Materia	Analisis y Diseno de Sistemas
Arquitecto	DDamianZR

37 Tablas	65+ Endpoints	9 Diagramas UML
16 Clases	12 Consultas SQL	8 Modulos

XOLIX 3.0

Documento de Diseño del Sistema

Proyecto: Plataforma web para la gestión integral de Niñas, Niños y Adolescentes (NNA) atendidos por una fundación dedicada a la restitución de derechos.

Versión: 3.0

Fecha: Junio 2026

Institución: Escuela Superior de Cómputo — Instituto Politécnico Nacional

Materia: Análisis y Diseño de Sistemas

Equipo de Desarrollo

Rol	Nombre
Arquitecto de Software	DDamianZR
Analista de Sistemas	—
Diseñador UML	—
Desarrollador Backend	—
Desarrollador Frontend	—

Historial de Revisiones

Versión	Fecha	Descripción
1.0	Feb 2026	Diseño inicial: autenticación, usuarios, expedientes
2.0	Mar 2026	Agregado familiograma interactivo y entrevistas
3.0	Jun 2026	Módulos completos: actores, diagnósticos, planes, auditoría, reportes

Tabla de Contenido

- Organización del Diseño
 - 1.1 Lineamientos de Diseño (SOLID, DRY, KISS, Clean Architecture)
 - 1.2 Metodología
 - 1.3 Notación UML

- 1.4 Alcance del Diseño
 - Diseño Arquitectónico
 - 2.1 Objetivos de la Arquitectura
 - 2.2 Diagrama Arquitectónico (Componentes + Despliegue)
 - 2.3 Explicación de Capas
 - 2.4 Beneficios y Limitaciones
 - Diseño Estático
 - 3.1 Descripción General
 - 3.2 Diseño de Subsistemas (Seguridad, Expedientes, Diagnósticos, Planeación, Actores, Familiograma)
 - 3.3 Diseño de Módulos
 - 3.4 Diseño de Paquetes
 - 3.5 Diseño de Clases (BCE: Boundary, Control, Entity)
 - Diseño Dinámico
 - 4.1 Login
 - 4.2 Crear Expediente
 - 4.3 Editar Expediente
 - 4.4 Registrar Diagnóstico
 - 4.5 Determinar Derechos Vulnerados
 - 4.6 Buscar Actores
 - 4.7 Crear Plan de Restitución
 - 4.8 Registrar Seguimiento
 - 4.9 Generar Familiograma
 - Diseño de Persistencia
 - 5.1 Modelo Relacional Completo
 - 5.2 Diccionario de Datos (15 tablas principales)
 - 5.3 Catálogo de Consultas (12 queries documentados)
 - 5.4 Índices y Restricciones
-

Resumen Ejecutivo

Xolix 3.0 es una plataforma web desarrollada para apoyar a una fundación dedicada a la atención y restitución de derechos de Niñas, Niños y Adolescentes (NNA) en situación de vulnerabilidad.

El sistema implementa una arquitectura en cinco capas: presentación (React 18 + Vite), controladores REST (FastAPI), servicios de negocio (Python 3.12), repositorio ORM (SQLAlchemy 2.0) y persistencia (PostgreSQL 18). La comunicación entre el frontend y el backend se realiza mediante una API REST documentada con OpenAPI/Swagger y protegida con JWT.

El diseño sigue los principios SOLID, el patrón Boundary-Control-Entity (BCE) y la arquitectura limpia (Clean Architecture), garantizando alta cohesión, bajo acoplamiento, seguridad por diseño y escalabilidad modular.

Módulos Implementados

Módulo	Descripción	Estado
Autenticación (JWT + RBAC)	Login con roles: director, coordinador, psicólogo, trabajador social, legal	■ Completo
Gestión de Usuarios	CRUD de personal con validación de RFC/CURP	■ Completo
Expedientes NNA	Caso, tutor, datos médicos, cartilla de vacunación	■ Completo
Familiograma Interactivo	Canvas ReactFlow, personas, relaciones familiares	■ Completo
Actores en Materia de Derechos	Directorio con servicios, horarios, filtros por municipio/derecho	■ Completo
Diagnósticos	4 tipos, evaluación de indicadores, derechos vulnerados automáticos	■ Completo
Planes de Restitución	Planes, medidas por tipo, seguimientos con avance	■ Completo
Auditoría	Registro automático de todas las operaciones críticas	■ Completo
Exportación	PDF y Excel de casos, actores y diagnósticos	■ Completo
Reportes e Indicadores	KPIs globales, frecuencia de derechos vulnerados, evolución mensual	■ Completo

Estadísticas del Sistema

Indicador	Valor
Tablas en base de datos	37
Endpoints de API REST	65+
Casos de uso documentados	9
Diagramas de secuencia UML	9
Clases del dominio	16
Clases BCE total	30+
Consultas SQL documentadas	12
Módulos de negocio	8

Instrucciones para Usar los Diagramas UML

Los diagramas de este documento están escritos en sintaxis PlantUML. Para visualizarlos:

Opción 1: Visual Paradigm / StarUML / Enterprise Architect

- Abrir la herramienta CASE

- Crear un nuevo diagrama del tipo correspondiente
- En el editor de código PlantUML, pegar el contenido del bloque `@startuml...@enduml`
- La herramienta generará el diagrama automáticamente

Opción 2: PlantUML Online

- Acceder a <https://www.plantuml.com/plantuml/uml/>
- Pegar el contenido del bloque entre `@startuml` y `@enduml`
- El diagrama se genera en tiempo real

Opción 3: VS Code con extensión PlantUML

- Instalar la extensión "PlantUML" de jebbs
- Abrir el archivo `.puml` o el bloque de código
- Presionar **Alt+D** para previsualizar

Opción 4: IntelliJ IDEA / WebStorm

- Instalar el plugin "PlantUML integration"
- Pegar el código en un archivo `.puml`
- El diagrama se genera en el panel lateral

Documento generado para el proyecto académico Xolix 3.0 — Análisis y Diseño de Sistemas — IPN ESCOM — 2026



ORGANIZACIÓN DEL DISEÑO

Principios SOLID · DRY · KISS · Clean Architecture · Metodología · Notación

1. ORGANIZACIÓN DEL DISEÑO

1.1 Lineamientos de Diseño

El diseño de Xolix 3.0 se rige por un conjunto de principios de ingeniería de software que garantizan la calidad, mantenibilidad y evolución del sistema a lo largo del tiempo. A continuación se detalla cada principio y su aplicación concreta en el sistema.

1.1.1 Principios SOLID

Los principios SOLID constituyen la base del diseño orientado a objetos de calidad. Su aplicación en Xolix 3.0 se describe a continuación:

S — Single Responsibility Principle (Principio de Responsabilidad Única)

Cada clase, módulo o componente debe tener una única razón para cambiar. En Xolix 3.0 esto se materializa de la siguiente manera:

- La clase **ExpedienteService** se ocupa exclusivamente de la lógica de negocio relacionada con el ciclo de vida del expediente (crear, actualizar, archivar). No tiene ninguna responsabilidad de presentación ni de persistencia directa.
- Los routers de FastAPI (**expedientes.py**, **diagnosticos.py**, **planes.py**) actúan únicamente como puntos de entrada HTTP; delegan toda la lógica a la capa de servicios.
- Los modelos SQLAlchemy (**CasoNNA**, **Diagnostico**, **PlanRestitucion**) son responsables exclusivamente del mapeo objeto-relacional.

O — Open/Closed Principle (Principio Abierto/Cerrado)

Las entidades de software deben estar abiertas para extensión y cerradas para modificación. En Xolix 3.0:

- El sistema de roles (**director**, **coordinador**, **psicologo**, **trabajador_social**, **legal**) se implementa mediante el decorador **require_role(*roles)** en **dependencies.py**. Para agregar un nuevo rol no es necesario modificar ningún router existente; basta con actualizar la lista de roles permitidos en la llamada al decorador.
- Las categorías de derechos se modelan como enumeraciones extensibles (**CategoriaDerecho**). Agregar una nueva categoría no modifica la lógica de validación existente.

L — Liskov Substitution Principle (Principio de Sustitución de Liskov)

Los objetos de una subclase deben poder usarse en lugar de objetos de su superclase sin alterar el comportamiento del programa. En Xolix 3.0:

- Los esquemas Pydantic de respuesta (**CasoNNAResponse**) extienden los esquemas base manteniendo compatibilidad total de tipos. Un endpoint que recibe **CasoNNACreate** puede ser reemplazado por **CasoNNAUpdate** sin romper el contrato de la API.
- Las enumeraciones de estado (**EstadoCasoNNA**, **EstadoPlan**) son subtipos de **str** en Python, lo que permite que sean tratadas como cadenas de texto ordinarias en cualquier contexto que espere un **str**.

I — Interface Segregation Principle (Principio de Segregación de Interfaces)

Los clientes no deben verse forzados a depender de interfaces que no utilizan. En Xolix 3.0:

- Los schemas Pydantic están separados por operación: **CasoNNACreate**, **CasoNNUpdate** y **CasoNNAResponse** son clases distintas. Un servicio que solo lee expedientes importa únicamente **CasoNNAResponse**, sin verse afectado por los campos requeridos en creación.
- Los routers están organizados por dominio de negocio, no por tipo de operación. Esto evita que un módulo cliente (por ejemplo, el de diagnósticos) dependa de las operaciones de planes.

D — Dependency Inversion Principle (Principio de Inversión de Dependencias)

Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones. En Xolix 3.0:

- Los servicios reciben la sesión de base de datos mediante inyección de dependencias de FastAPI (**db: Session = Depends(get_db)**). El servicio no instancia ni gestiona directamente la conexión a PostgreSQL.
- Las dependencias de autenticación (**get_current_user**) y autorización (**require_role**) se inyectan en los endpoints, no se invocan directamente. Esto permite sustituirlas en pruebas sin modificar la lógica de negocio.

1.1.2 DRY — Don't Repeat Yourself

El principio DRY establece que cada pieza de conocimiento debe tener una representación única, no ambigua y autoritativa dentro de un sistema.

En Xolix 3.0:

- La validación de CURP y RFC se centraliza en **validators/mexican_ids.py**. Ningún router ni servicio repite esta lógica.
- El cliente HTTP del frontend se encapsula en **src/api/client.js**, que gestiona la adición del token JWT en cada petición. Los componentes React no repiten la lógica de autenticación en cada llamada.
- La función **registrar_audit()** en **extras_service.py** centraliza el registro de auditoría. Todos los servicios que necesitan registrar acciones la llaman en lugar de escribir directamente en la tabla **audit_logs**.
- Los estilos de la interfaz se definen mediante variables CSS globales en **index.css** (**--primary**, **--neo-shadow**, **--radius-sm**). Los componentes referencian las variables, no valores hexadecimales repetidos.

1.1.3 KISS — Keep It Simple, Stupid

El principio KISS establece que la mayoría de los sistemas funcionan mejor si se mantienen simples en lugar de complejos.

En Xolix 3.0:

- Los endpoints REST siguen convenciones estándar de HTTP (GET para consultas, POST para creación, PUT para actualización, DELETE para eliminación). No se inventan convenciones propietarias.
- La autenticación se basa en JWT estándar (RFC 7519) sin capas adicionales de complejidad. El token contiene únicamente el **user_id** y el **rol**, que son los datos mínimos necesarios.
- El familiograma se almacena como JSON en PostgreSQL en lugar de modelar un grafo relacional complejo. Esto simplifica drásticamente las operaciones de lectura y escritura, que son las más

frecuentes para el familiograma.

1.1.4 Clean Architecture

La arquitectura limpia (Robert C. Martin) establece que las reglas de negocio son el núcleo del sistema y no deben depender de frameworks, bases de datos o interfaces de usuario.

En Xolix 3.0 las capas son:



La regla de dependencia se respeta: las entidades de dominio no importan nada de FastAPI; los servicios no importan nada de los routers; los modelos no importan nada de los schemas.

1.1.5 Baja Dependencia (Bajo Acoplamiento)

El acoplamiento se minimiza mediante:

- **Inyección de dependencias:** FastAPI provee `Depends()` para inyectar sesiones de BD y dependencias de seguridad sin acoplar los handlers al ciclo de vida de la aplicación.
- **Separación de modelos ORM y schemas:** los modelos SQLAlchemy nunca se serializan directamente a JSON; los schemas Pydantic actúan como capa de transformación, evitando que cambios en la BD afecten directamente la API.
- **API REST:** el frontend y el backend están desacoplados por contrato HTTP. El frontend puede desarrollarse o probarse independientemente.

1.1.6 Alta Cohesión

Cada módulo agrupa únicamente lo que pertenece al mismo dominio conceptual:

- El paquete `app/models/diagnostico.py` contiene `Diagnostico`, `IndicadorDiagnostico` y `DerechoVulnerado`: entidades que solo tienen sentido en el contexto del diagnóstico.
- El paquete `app/routers/planes.py` expone únicamente los endpoints relacionados con planes de restitución.
- Los componentes React `DiagnosticoPage.jsx` y `PlanesPage.jsx` son responsables exclusivamente de su dominio funcional.

1.1.7 Seguridad por Diseño

- **Autenticación:** JWT con expiración configurable (`ACCESS_TOKEN_EXPIRE_MINUTES`).

- **Autorización:** RBAC implementado mediante el decorador `require_role(*roles)` aplicado a nivel de endpoint.
- **Hashing de contraseñas:** bcrypt con factor de costo configurable.
- **Variables de entorno:** credenciales en `.env`, nunca en código fuente.
- **Validación de entrada:** Pydantic valida todos los datos antes de que lleguen a la capa de servicios.
- **Auditoría:** todas las operaciones críticas quedan registradas en `audit_logs` con usuario, acción, entidad e identidad de registro.

1.1.8 Escalabilidad

- **Horizontal:** al ser una aplicación sin estado (stateless) con JWT, múltiples instancias del backend pueden ejecutarse simultáneamente detrás de un balanceador.
- **Vertical:** SQLAlchemy gestiona un pool de conexiones configurable a PostgreSQL.
- **Modular:** cada módulo de negocio (Expedientes, Diagnósticos, Actores, Planes) puede evolucionar o migrarse independientemente.

1.1.9 Reutilización

- Los componentes React (**Topbar**, tarjetas de estado, formularios reutilizables) están diseñados como unidades independientes con props bien definidas.
- Los servicios de exportación (`export_service.py`) son reutilizables por cualquier módulo que necesite generar PDF o Excel.
- El catálogo de derechos e indicadores es compartido por el módulo de diagnósticos y el de planes.

1.2 Metodología

1.2.1 UML 2.x

El diseño utiliza la versión 2.x del Unified Modeling Language según el estándar de la Object Management Group (OMG). Los diagramas producidos son compatibles con herramientas CASE profesionales como Visual Paradigm, StarUML y Enterprise Architect.

Los tipos de diagrama utilizados en este documento son:

Tipo de Diagrama	Estándar UML	Propósito
Diagrama de Componentes	UML 2.x §15	Arquitectura del sistema
Diagrama de Despliegue	UML 2.x §19	Infraestructura física/virtual
Diagrama de Clases	UML 2.x §9	Estructura estática del dominio
Diagrama de Paquetes	UML 2.x §12	Organización modular
Diagrama de Secuencia	UML 2.x §17	Comportamiento dinámico

1.2.2 Diseño Orientado a Objetos

El diseño aplica los conceptos fundamentales del paradigma orientado a objetos:

- **Abstracción:** las clases del dominio (**CasoNNA**, **Diagnostico**) modelan conceptos del mundo real de la gestión de NNA, ocultando los detalles de implementación.
- **Encapsulamiento:** los atributos de las entidades son privados; el acceso se controla mediante métodos de servicio.
- **Herencia:** los schemas Pydantic usan herencia (**CasoNNAUpdate** hereda de **CasoNNACreate**) para reutilizar validaciones.
- **Polimorfismo:** los distintos tipos de diagnóstico (**inicial**, **nna**, **tutor**, **entorno**) comparten la misma interfaz de gestión.

1.2.3 Casos de Uso

Los casos de uso se modelan siguiendo la notación UML estándar e identifican las interacciones entre los actores del sistema (Director, Coordinador, Psicólogo, Trabajador Social, Legal) y las funcionalidades del sistema.

Los casos de uso cubren los nueve procesos principales descritos en la Sección 4 (Diseño Dinámico).

1.2.4 Desarrollo Iterativo e Incremental

El proyecto se desarrolló en tres iteraciones:

Iteración	Módulos entregados	Estado
1	Usuarios, Expedientes, Procesos	Completada
2	Familiograma (interactivo), Personas, Relaciones	Completada
3	Actores, Diagnósticos, Planes, Auditoría, Reportes	Completada

Cada iteración produjo un incremento funcional y demostrable, siguiendo el ciclo: Análisis → Diseño → Implementación → Prueba → Integración.

1.3 Notación

1.3.1 Diagrama de Clases UML

Elemento	Notación	Significado
+atributo: Tipo	Visibilidad pública	Atributo público
-atributo: Tipo	Visibilidad privada	Atributo privado
#atributo: Tipo	Visibilidad protegida	Atributo protegido
■ ■ ■ ■ ■ >	Asociación	Relación estructural
◆ ■ ■ ■ ■ ■	Composición	El todo controla el ciclo de vida
■ ■ ■ ■ ■ ■	Agregación	Asociación "parte de" débil
■ ■ ■ ■ ■ ■	Herencia/Generalización	Subclase → Superclase

Elemento	Notación	Significado
- - - ->	Dependencia	Uso transitorio
1, 0..1, *, 1..*	Multiplicidad	Cardinalidad de la relación

1.3.2 Diagrama de Secuencia UML

Elemento	Notación	Significado
:Actor	Figura de palo + lifeline	Agente externo
:IUxxx	Rectángulo + lifeline	Clase Boundary
:xxxCtr	Círculo con flecha + lifeline	Clase Control
:ORMxxx	Elipse + lifeline	Clase Entity
:PostgreSQL	Cilindro + lifeline	Base de datos
■■■■■>	Mensaje síncrono	Llamada bloqueante
■■■■■>>	Mensaje asíncrono	Llamada no bloqueante
<■■■■■	Retorno	Valor de retorno
■	Barra de activación	Objeto en ejecución
loop[cond]	Fragmento combinado	Iteración
alt[cond]	Fragmento combinado	Alternativa (if/else)
opt[cond]	Fragmento combinado	Opcional (if sin else)
ref	Fragmento de referencia	Referencia a otro diagrama

1.3.3 Patrón BCE (Boundary-Control-Entity)

El patrón BCE organiza las clases de un caso de uso en tres categorías:

Categoría	Estereotipo	Responsabilidad
Boundary	<<boundary>>	Interfaz con actores externos (UI, API). Traduce entre el mundo externo y el sistema.
Control	<<control>>	Orquesta la lógica del caso de uso. Coordina Boundaries y Entities.
Entity	<<entity>>	Representa objetos de dominio persistentes. Encapsula datos y reglas de negocio.

En la implementación Xolix 3.0, el mapeo es:

```
Boundary ↔ Router FastAPI / Componente React
Control  ↔ Service (expediente_service.py, diagnostico_service.py...)
Entity   ↔ Modelo SQLAlchemy (CasoNNA, Diagnostico, Actor...)
```

1.3.4 Diagrama de Paquetes UML

Los paquetes se representan con la notación de pestaña UML. Las dependencias entre paquetes se muestran con flechas punteadas (- - →). Un paquete que importa a otro tiene una dependencia hacia él.

1.3.5 Modelo Relacional

El modelo relacional se representa con notación crow's foot (pata de gallo):

Símbolo	Significado
■ ■ ■	Uno (obligatorio)
o ■ ■	Cero o uno (opcional)
> ■ ■	Muchos
> o ■ ■	Cero o muchos

1.4 Alcance del Diseño

1.4.1 Lo que cubre este documento

El presente Documento de Diseño cubre los siguientes aspectos del sistema Xolix 3.0:

- **Diseño arquitectónico:** estructura en capas, componentes y sus responsabilidades.
- **Diseño estático:** estructura de clases, subsistemas, módulos, paquetes y sus interrelaciones.
- **Diseño dinámico:** comportamiento del sistema en los nueve casos de uso principales mediante diagramas de secuencia.
- **Diseño de persistencia:** modelo relacional completo, diccionario de datos y catálogo de consultas SQL.
- **Aplicación del patrón BCE** en todos los diagramas dinámicos.
- **Decisiones de diseño justificadas:** cada decisión significativa se acompaña de una justificación técnica.

1.4.2 Lo que NO cubre este documento

- **Diseño de pruebas:** las estrategias de pruebas unitarias, de integración y de aceptación se documentan en el Plan de Pruebas (documento independiente).
- **Diseño de infraestructura de producción:** balanceadores de carga, contenedores Docker, pipelines CI/CD y configuración de servidores en producción se describen en el documento de despliegue.
- **Interfaz móvil:** Xolix 3.0 es exclusivamente una aplicación web responsiva. No existe versión móvil nativa en el alcance actual.
- **Módulo de Business Intelligence:** los reportes actuales son operacionales. Los dashboards analíticos avanzados (OLAP, predicciones) quedan fuera del alcance.
- **Integración con sistemas externos:** en este momento el sistema no se integra con sistemas de registro civil, IMSS, DIF ni otras plataformas gubernamentales.

1.4.3 Versión del sistema cubierta

Este documento corresponde a la versión **3.0** de Xolix, que incluye los módulos:

- Gestión de Usuarios y Autenticación (v1.0)
- Expedientes Digitales (v1.0)
- Procesos/Tareas (v1.0)
- Familiograma Interactivo (v2.0)
- Actores en Materia de Derechos (v3.0)
- Diagnósticos y Derechos Vulnerados (v3.0)
- Planes de Restitución y Seguimientos (v3.0)
- Auditoría y Exportación (v3.0)
- Reportes e Indicadores Globales (v3.0)



DISEÑO ARQUITECTÓNICO

[Objetivos](#) · [Diagrama de Componentes](#) · [Capas](#) · [Beneficios y Limitaciones](#)

2. DISEÑO ARQUITECTÓNICO

2.1 Objetivo de la Arquitectura

La arquitectura de Xolix 3.0 ha sido diseñada para satisfacer los siguientes atributos de calidad, derivados directamente de los requisitos no funcionales del sistema:

2.1.1 Mantenibilidad

Objetivo: el sistema debe poder modificarse para corregir defectos, agregar funcionalidades o adaptarse a nuevos requerimientos con un costo de cambio mínimo.

Decisiones arquitectónicas:

- Separación estricta en capas (Presentación → Servicios → Persistencia). Un cambio en la base de datos no afecta la lógica de negocio; un cambio en la interfaz no afecta los servicios.
- Cada módulo de negocio (Expedientes, Diagnósticos, Actores, Planes) tiene su propio conjunto de modelos, schemas, servicios y routers. Un error en el módulo de Diagnósticos no propaga cambios al módulo de Planes.
- El uso de Pydantic para la validación centraliza las reglas de validación en un único lugar, facilitando su localización y modificación.

2.1.2 Escalabilidad

Objetivo: el sistema debe poder atender un número creciente de usuarios y volumen de datos sin necesidad de rediseño.

Decisiones arquitectónicas:

- **Escalabilidad horizontal del backend:** FastAPI es una aplicación ASGI sin estado. Múltiples instancias pueden ejecutarse en paralelo detrás de un proxy reverso (Nginx) o balanceador de carga, ya que el estado de sesión se gestiona mediante JWT, no mediante sesiones en servidor.
- **Pool de conexiones:** SQLAlchemy gestiona un pool de conexiones a PostgreSQL, evitando la sobrecarga de establecer una nueva conexión por cada petición.
- **Separación de frontend y backend:** el servidor de React/Vite puede servirse desde una CDN sin escalar el backend.

2.1.3 Disponibilidad

Objetivo: el sistema debe estar disponible durante las horas laborales de la fundación con un tiempo de recuperación ante fallas bajo.

Decisiones arquitectónicas:

- La arquitectura permite despliegue en contenedores (Docker), facilitando reinicio automático ante fallas.
- PostgreSQL soporta replicación para alta disponibilidad.
- Las migraciones de base de datos son versionadas, permitiendo rollback controlado.

2.1.4 Seguridad

Objetivo: el acceso a la información de los NNA debe estar estrictamente controlado por identidad y rol.

Decisiones arquitectónicas:

- **Autenticación JWT:** todos los endpoints (excepto `/api/auth/login`) requieren un token válido. El token tiene tiempo de expiración configurable.
- **Control de acceso basado en roles (RBAC):** el decorador `require_role(*roles)` en FastAPI verifica el rol del usuario antes de procesar cualquier operación.
- **Validación de entrada en múltiples capas:** Pydantic valida en la capa de API; SQLAlchemy aplica restricciones en la capa de persistencia (NOT NULL, UNIQUE, FK).
- **Contraseñas con bcrypt:** las contraseñas se almacenan como hash irreversible. Nunca se almacenan en texto plano.
- **Auditoría activa:** cada operación de escritura (crear, modificar, eliminar) se registra en `audit_logs` con el ID del usuario, la acción y los datos afectados.

2.1.5 Modularidad

Objetivo: el sistema debe poder extenderse con nuevos módulos sin afectar los existentes.

Decisiones arquitectónicas:

- Los módulos se registran en `main.py` mediante `app.include_router()`. Agregar un nuevo módulo requiere únicamente crear sus archivos de modelo/schema/servicio/router y registrarlo.
- El catálogo de derechos e indicadores es independiente de los módulos que lo consumen (Diagnósticos, Planes), comunicándose mediante IDs.

2.1.6 Rendimiento

Objetivo: las páginas deben cargarse en menos de 3 segundos bajo carga normal; las consultas de listado en menos de 1 segundo.

Decisiones arquitectónicas:

- Índices en columnas de búsqueda frecuente (`caso_nna_id`, `responsable_id`, `estado`).
- Paginación disponible en todos los endpoints de listado para limitar el volumen de datos transferidos.
- React con Vite produce bundles optimizados con code splitting automático.
- Los endpoints de exportación (PDF/Excel) usan `StreamingResponse` para no bloquear el servidor durante la generación.

2.2 Diagrama Arquitectónico

2.2.1 Diagrama de Componentes UML 2.x

```
@startuml
    Xolix_Arquitectura_Componentes
    skinparam componentStyle uml2
    skinparam backgroundColor #FAFAFA
    skinparam component {
        BackgroundColor #E8F4FD
        BorderColor #2980B9
    }
}
```



```

[ORMCasonNA]    --> [SessionFactory]
[ORMDiagnostico]--> [SessionFactory]
[ORMPlan]       --> [SessionFactory]
[SessionFactory]--> DB

[JWT Middleware] --> [python-jose]
[UserService]    --> [bcrypt]
[ExportService]  --> [reportlab]
[ExportService]  --> [openpyxl]
[Router Auth]    --> [zippopotam.us API] : HTTP (sepomex)

@enduml

```

2.2.2 Diagrama de Despliegue UML 2.x

```

@startuml Xolix_Despliegue

skinparam nodeStyle uml2
skinparam backgroundColor #FAFAFA

title Diagrama de Despliegue – Xolix 3.0\nEntorno de Desarrollo Local

node "Estación de Trabajo\n[Windows 11]" as WS {
    node "Navegador Web\n[Chrome / Firefox]" as Browser {
        artifact "React SPA\n[xolix-frontend.js]" as SPA
    }

    node "Servidor Frontend\n[Node.js + Vite Dev Server\npuerto 5173]" as FrontendServer {
        artifact "Frontend Bundle\n[React 18 + JSX]" as FrontendApp
    }

    node "Servidor Backend\n[Python 3.12 + Uvicorn\npuerto 8000]" as BackendServer {
        artifact "FastAPI App\n[app.main:app]" as FastAPIApp
        artifact "SQLAlchemy ORM\n[engine + session]" as ORM
    }

    node "Servidor de Base de Datos\n[PostgreSQL 18\npuerto 5432]" as DBServer {
        database "proyecto_escom" as DB {
            artifact "37 tablas relacionales" as Tables
        }
    }
}

node "API Externa\n[zippopotam.us]" as ExtAPI

Browser      --> FrontendServer : HTTP :5173
Browser      --> BackendServer  : REST API :8000 (JSON + JWT)
FrontendServer --> FrontendApp
BackendServer --> FastAPIApp
FastAPIApp    --> ORM
ORM           --> DB              : SQL (psycopg2)
FastAPIApp    --> ExtAPI          : HTTP (código postal)

@enduml

```

2.3 Explicación de la Arquitectura

2.3.1 Capa de Presentación (Frontend)

Tecnología: React 18 con Vite 5, React Router v6, @xyflow/react.

Responsabilidades:

- Renderizar la interfaz de usuario en el navegador del usuario.
- Gestionar el estado local de la aplicación (useState, useEffect).
- Realizar peticiones HTTP a la API REST a través del cliente centralizado `src/api/client.js`.
- Almacenar el JWT en `localStorage` y adjuntarlo en el encabezado **Authorization: Bearer** de cada petición.
- Gestionar la navegación entre módulos mediante React Router.

Componentes principales:

- `pages/`: páginas completas, una por funcionalidad principal.
- `components/`: componentes reutilizables (Topbar, formularios, tarjetas).
- `api/client.js`: abstracción del cliente HTTP con gestión automática de token.
- `context/`: contexto React para el estado global del usuario autenticado.

Dependencias de entrada: ninguna (es la capa más externa).

Dependencias de salida: únicamente la API REST del backend.

2.3.2 Capa de API REST (Backend — Routers FastAPI)

Tecnología: FastAPI 0.111 sobre ASGI (Uvicorn).

Responsabilidades:

- Exponer los endpoints HTTP de la API REST.
- Validar la estructura y tipos de los datos de entrada mediante schemas Pydantic.
- Verificar la autenticación (JWT válido) y la autorización (rol requerido) en cada endpoint.
- Deserializar las peticiones y serializar las respuestas en JSON.
- Delegar la ejecución de la lógica de negocio a la capa de servicios.

Restricciones:

- Los routers NO ejecutan lógica de negocio directamente. Solo llaman a funciones de la capa de servicios.
- Los routers NO acceden directamente a la base de datos. Toda la persistencia pasa por SQLAlchemy a través de los servicios.

Middleware aplicado:

- **CORSMiddleware**: permite peticiones desde `http://localhost:5173` (frontend de desarrollo).
- **JWT Middleware**: verifica el token en cada petición mediante `get_current_user()`.
- **RBAC Dependency**: verifica el rol del usuario mediante `require_role(*roles)`.

2.3.3 Capa de Servicios (Lógica de Negocio)

Tecnología: Python 3.12 puro (sin dependencias de framework).

Responsabilidades:

- Implementar las reglas de negocio del dominio.

- Orquestar operaciones que involucran múltiples entidades (ejemplo: crear un diagnóstico genera automáticamente los registros de derechos vulnerados).
- Gestionar transacciones de base de datos (commit/rollback).
- Registrar las acciones en el log de auditoría.

Restricciones:

- Los servicios NO importan nada de FastAPI (ni Request, ni Response, ni HTTPException en algunas versiones).
- Los servicios reciben la sesión de BD como parámetro (**db: Session**), nunca la crean directamente.

2.3.4 Capa ORM / Repositorio (SQLAlchemy)

Tecnología: SQLAlchemy 2.0 con driver psycopg2.

Responsabilidades:

- Mapear las clases Python a tablas de PostgreSQL mediante el patrón Active Record.
- Gestionar el pool de conexiones a la base de datos.
- Traducir las operaciones de Python (consultas, inserciones) a SQL.
- Gestionar las relaciones entre entidades mediante **relationship()**.

Restricciones:

- Los modelos ORM NO contienen lógica de negocio compleja (solo validaciones simples del dominio).
- Las tablas se crean automáticamente con **Base.metadata.create_all(bind=engine)** al iniciar la aplicación.

2.3.5 Capa de Persistencia (PostgreSQL)

Tecnología: PostgreSQL 18.

Responsabilidades:

- Almacenar todos los datos del sistema de forma persistente y con integridad referencial.
- Ejecutar las consultas SQL generadas por SQLAlchemy.
- Aplicar restricciones de integridad (PK, FK, NOT NULL, UNIQUE, CHECK).
- Gestionar la concurrencia mediante transacciones ACID.

Base de datos: proyecto_escom con 37 tablas relacionales.

2.4 Beneficios y Limitaciones

2.4.1 Beneficios de la Arquitectura

Beneficio	Descripción
Desacoplamiento total frontend/backend	El frontend y el backend pueden desplegarse, escalarse y desarrollarse de forma completamente independiente. El contrato es únicamente la API REST documentada con OpenAPI/Swagger.

Beneficio	Descripción
Documentación automática de la API	FastAPI genera automáticamente la documentación Swagger UI (/docs) y ReDoc (/redoc) a partir del código.
Tipado fuerte en dos lenguajes	Python con Pydantic + TypeScript (opcional) en el frontend proporciona validación de tipos en tiempo de ejecución y en compilación.
Rapidez de desarrollo	FastAPI reduce significativamente el boilerplate comparado con Django REST Framework o Flask.
Testabilidad	La inyección de dependencias de FastAPI facilita reemplazar la sesión de BD por una de prueba sin modificar el código de producción.
Extensibilidad modular	Agregar un nuevo módulo de negocio requiere únicamente crear los archivos correspondientes y registrar el router. No se modifican archivos existentes.

2.4.2 Limitaciones de la Arquitectura

Limitación	Descripción	Mitigación
Sin caché distribuido	No hay Redis u otro sistema de caché. Consultas frecuentes (catálogo de derechos, lista de actores) se ejecutan contra la BD en cada petición.	Agregar Redis en iteraciones futuras para caché de catálogos.
Sin búsqueda full-text optimizada	Las búsquedas de texto usan ILIKE de PostgreSQL, que no escala bien con millones de registros.	Integrar PostgreSQL Full Text Search o Elasticsearch en versiones futuras.
Sin WebSockets	Las notificaciones en tiempo real (por ejemplo, cuando se actualiza el estado de un caso) requieren polling del frontend.	Implementar Server-Sent Events o WebSockets con FastAPI en versiones futuras.
JWT sin revocación	Un JWT emitido es válido hasta su expiración incluso si el usuario cambia su contraseña o es desactivado.	Implementar una lista negra de tokens (blacklist) en Redis en versiones futuras.
SQLite no compatible	El sistema está diseñado específicamente para PostgreSQL (usa tipos JSON, ENUM nativos). No puede migrarse trivialmente a SQLite o MySQL.	Esta limitación es aceptable dado el entorno de despliegue objetivo.



DISEÑO ESTÁTICO

Subsistemas · Módulos · Paquetes · Clases BCE (Boundary · Control · Entity)

3. DISEÑO ESTÁTICO

3.1 Descripción General

El diseño estático de Xolix 3.0 describe la estructura del software: cómo se organizan los componentes, qué clases existen, cómo se relacionan entre sí y cómo se agrupan en paquetes y subsistemas.

El sistema se divide en **seis subsistemas funcionales** que corresponden directamente a los módulos de negocio: Seguridad, Expedientes, Diagnósticos, Planeación, Actores y Familiograma. Cada subsistema opera de forma relativamente independiente, comunicándose a través de interfaces bien definidas.

La estructura sigue el patrón BCE (Boundary-Control-Entity) para organizar las responsabilidades dentro de cada subsistema. Las clases de presentación (Boundary) no conocen los detalles de persistencia; las clases de control (servicios) coordinan sin acoplarse a los detalles de la interfaz; las clases de entidad modelan el dominio sin conocer a sus consumidores.

3.2 Diseño de Subsistemas

3.2.1 Subsistema de Seguridad

Responsabilidades:

- Autenticar usuarios mediante usuario y contraseña.
- Generar y validar tokens JWT.
- Verificar el rol del usuario en cada petición (RBAC).
- Registrar intentos de acceso en el log de auditoría.

Interfaces expuestas:

- **POST** /api/auth/login → retorna {access_token, token_type}
- **GET** /api/auth/me → retorna los datos del usuario autenticado

Dependencias:

- Depende del subsistema de Expedientes únicamente para cargar el perfil del usuario.
- Provee autenticación a todos los demás subsistemas mediante la dependencia `get_current_user`.

Clases participantes:

- Boundary: **IULogin** (LoginPage.jsx)
- Control: **AccessCtr** (auth.py router + security.py)
- Entity: **ORMUser** (models/user.py)

3.2.2 Subsistema de Expedientes

Responsabilidades:

- Gestionar el ciclo de vida completo del caso NNA: creación, actualización, cierre.
- Administrar los datos personales, médicos, familiares y del tutor del NNA.

- Gestionar la carga de documentos del expediente.
- Proveer funcionalidades de búsqueda y filtrado de casos.

Interfaces expuestas:

- **GET/POST /api/nna/casos** → listar y crear casos
- **GET/PUT /api/nna/casos/{id}** → obtener y actualizar caso
- **GET/PUT /api/nna/casos/{id}/tutor** → gestionar tutor
- **GET/PUT /api/nna/casos/{id}/datos-medicos** → gestionar datos médicos

Dependencias:

- Depende del Subsistema de Seguridad para autenticación y roles.
- Provee la entidad **CasoNNA** al Subsistema de Diagnósticos y al de Planeación.

Clases participantes:

- Boundary: **IUExpediente** (NuevoCasoNNA.jsx, CasoNNADetalle.jsx)
- Control: **ExpedienteCtr** (nna_service.py, nna.py router)
- Entity: **ORMCasoNNA**, **ORMTutorNNA**, **ORMDatosMedicosNNA**

3.2.3 Subsistema de Diagnósticos

Responsabilidades:

- Registrar diagnósticos de cuatro tipos: inicial, NNA, tutor y entorno.
- Evaluar indicadores de derechos para cada diagnóstico.
- Generar automáticamente los registros de derechos vulnerados cuando un indicador es marcado como vulnerado.
- Proveer el resumen de derechos vulnerados por caso para su uso en planeación.

Interfaces expuestas:

- **GET/POST /api/diagnosticos** → listar y crear diagnósticos
- **GET /api/diagnosticos/{id}** → detalle de diagnóstico
- **GET /api/diagnosticos/caso/{id}/derechos-vulnerados** → resumen por caso

Dependencias:

- Depende del Subsistema de Expedientes para la entidad **CasoNNA**.
- Depende del catálogo de **Derecho e Indicador**.
- Provee los derechos vulnerados al Subsistema de Planeación.

Clases participantes:

- Boundary: **IUDiagnostico** (DiagnosticoPage.jsx)
- Control: **DiagnosticoCtr** (diagnostico_service.py, diagnosticos.py router)
- Entity: **ORMDiagnostico**, **ORMIndicadorDiagnostico**, **ORMDerechoVulnerado**

3.2.4 Subsistema de Planeación

Responsabilidades:

- Crear planes de restitución con objetivos, responsables y fechas.

- Registrar medidas de restitución de diferentes tipos (psicológica, legal, médica, educativa, social, económica).
- Registrar seguimientos de avance para cada medida.
- Actualizar automáticamente el porcentaje de avance de la medida al registrar seguimientos.

Interfaces expuestas:

- **GET/POST /api/planes** → listar y crear planes
- **GET/PUT /api/planes/{id}** → detalle y actualización de plan
- **POST /api/planes/{id}/medidas** → agregar medida
- **POST /api/planes/medidas/{id}/seguimientos** → registrar seguimiento

Dependencias:

- Depende del Subsistema de Expedientes para la entidad **CasoNNA**.
- Depende del Subsistema de Diagnósticos para identificar los derechos afectados.
- Depende del Subsistema de Actores para vincular medidas con actores.

Clases participantes:

- Boundary: **IUPlan** (PlanesPage.jsx)
- Control: **PlanCtr** (plan_service.py, planes.py router)
- Entity: **ORMPlanRestitucion**, **ORMMedidaRestitucion**, **ORMSeguimientoMedida**

3.2.5 Subsistema de Actores

Responsabilidades:

- Registrar actores (organizaciones gubernamentales, civiles, empresas, personas físicas).
- Gestionar los datos de contacto, horarios, servicios y requisitos de cada actor.
- Proveer búsqueda de actores por municipio, derecho vulnerado y tipo de servicio.
- Vincular servicios de actores con derechos del catálogo.

Interfaces expuestas:

- **GET/POST /api/actores** → listar y crear actores (con filtros de búsqueda)
- **GET/PUT /api/actores/{id}** → detalle y actualización
- **POST /api/actores/{id}/servicios** → agregar servicio

Dependencias:

- Depende del Catálogo de Derechos para vincular servicios.
- Es consumido por el Subsistema de Planeación para vincular medidas con actores.

Clases participantes:

- Boundary: **IUActor** (ActoresList.jsx, ActorDetalle.jsx, ActorForm.jsx)
- Control: **ActorCtr** (actor_service.py, actores.py router)
- Entity: **ORMActor**, **ORMResponsableActor**, **ORMServicioActor**, **ORMHorarioActor**

3.2.6 Subsistema de Familiograma

Responsabilidades:

- Registrar personas del entorno familiar del NNA.

- Registrar relaciones entre personas del entorno familiar.
- Gestionar el canvas visual del familiograma mediante ReactFlow.
- Almacenar el grafo como JSON en PostgreSQL.
- Mantener historial de versiones del familiograma.

Interfaces expuestas:

- **GET/POST** /api/nna/casos/{id}/personas → personas del caso
- **GET/POST** /api/nna/casos/{id}/relaciones → relaciones familiares
- **GET/PUT** /api/nna/casos/{id}/familiograma → canvas del grafo

Dependencias:

- Depende del Subsistema de Expedientes para la entidad **CasoNNA**.

Clases participantes:

- Boundary: **IUFamiliograma** (FamiliogramaEditor.jsx, PersonasFamiliaresPage.jsx)
- Control: **FamiliogramaCtr** (nna_service.py — funciones de familiograma)
- Entity: **ORMPersonaFamiliar**, **ORMRelacionFamiliar**, **ORMFamiliograma**

3.3 Diseño de Módulos

3.3.1 Módulo de Autenticación (auth)

Atributo	Detalle
Objetivo	Gestionar la identidad de los usuarios del sistema
Entradas	Credenciales (correo, contraseña) en JSON
Salidas	Token JWT, datos del perfil del usuario
Dependencias	security.py (hash/verify), user_service.py , PostgreSQL tabla users

3.3.2 Módulo de Usuarios (users)

Atributo	Detalle
Objetivo	Administrar el personal de la fundación (CRUD)
Entradas	Datos del empleado (nombre, RFC, CURP, rol, etc.)
Salidas	Perfil de usuario, lista paginada de usuarios
Dependencias	auth , validadores de RFC/CURP, tabla users

3.3.3 Módulo de Casos NNA (nna)

Atributo	Detalle
Objetivo	Gestionar el expediente completo del NNA

Atributo	Detalle
Entradas	Datos del NNA, tutor, datos médicos, vacunación
Salidas	Expediente completo, lista de casos por estado
Dependencias	auth, familiograma , tablas nna_casos, nna_tutores, nna_datos_medicos

3.3.4 Módulo de Catálogo (catalogo)

Atributo	Detalle
Objetivo	Gestionar el catálogo de derechos e indicadores de evaluación
Entradas	Datos del derecho (nombre, categoría, artículo) e indicadores
Salidas	Listado de derechos por categoría, indicadores por derecho
Dependencias	Tablas derechos, indicadores

3.3.5 Módulo de Actores (actores)

Atributo	Detalle
Objetivo	Registrar y localizar organizaciones y personas que pueden apoyar la restitución de derechos
Entradas	Datos del actor, servicios, horarios, responsables
Salidas	Directorio de actores filtrable, detalle con servicios y requisitos
Dependencias	catalogo (derechos), tablas actores, actores_servicios, actores_horarios

3.3.6 Módulo de Diagnóstico (diagnosticos)

Atributo	Detalle
Objetivo	Documentar la evaluación del NNA, tutor y entorno; identificar derechos vulnerados
Entradas	Tipo de diagnóstico, evaluación de indicadores, observaciones
Salidas	Diagnóstico con derechos vulnerados generados automáticamente
Dependencias	nna (caso), catalogo (indicadores/derechos), tablas diagnosticos, indicadores_diagnostico, derechos_vulnerados

3.3.7 Módulo de Planes (planes)

Atributo	Detalle
Objetivo	Planificar y hacer seguimiento de las medidas de restitución de derechos
Entradas	Objetivo del plan, medidas (tipo, responsable, actor), seguimientos de avance
Salidas	Plan con medidas y avances, porcentaje global de cumplimiento
Dependencias	nna (caso), actores , diagnosticos (derechos afectados), tablas planes_restitucion , medidas_restitucion , seguimientos_medida

3.3.8 Módulo de Reportes (reportes)

Atributo	Detalle
Objetivo	Generar indicadores globales, reportes de evolución y exportaciones PDF/Excel
Entradas	Parámetros de filtrado (fecha, estado, tipo)
Salidas	KPIs, gráficas de derechos vulnerados, archivos PDF/Excel
Dependencias	Todos los módulos (consume datos de todas las tablas), report lab , openpyxl

3.4 Diseño de Paquetes

3.4.1 Estructura de Paquetes Backend

```
@startuml
Xolix_Paquetes_Backend

skinparam packageStyle rectangle
skinparam backgroundColor #FAFAFA

title Diagrama de Paquetes – Backend Xolix 3.0

package "app" <<module>> {

    package "app.models" <<module>> {
        package "app.models.user" { class User }
        package "app.models.caso" { class Caso }
        package "app.models.nna" { class CasoNNA; class TutorNNA; class DatosMedicosNNA }
        package "app.models.catalogo" { class Derecho; class Indicador }
        package "app.models.actor" { class Actor; class ServicioActor }
        package "app.models.diagnostico" { class Diagnostico; class DerechoVulnerado }
        package "app.models.plan" { class PlanRestitucion; class MedidaRestitucion }
        package "app.models.extras" { class AuditLog; class Comentario }
        package "app.models.proceso" { class Proceso; class Subtarea }
        package "app.models.expediente" { class Expediente }
    }

    package "app.schemas" <<module>> {
        package "app.schemas.user" { class UserCreate; class UserResponse }
        package "app.schemas.nna" { class CasoNNACreate; class CasoNNAResponse }
    }
}
```

```
package "app.schemas.catalogo"      { class DerechoCreate; class DerechoResponse }
package "app.schemas.actor"        { class ActorCreate; class ActorResponse }
package "app.schemas.diagnostico"  { class DiagnosticoCreate; class DiagnosticoResponse }
package "app.schemas.plan"         { class PlanCreate; class PlanResponse }
}

package "app.services" <<module>> {
  package "app.services.user_service"      { class UserService }
  package "app.services.nna_service"       { class NNAService }
  package "app.services.catalogo_service"  { class CatalogoService }
  package "app.services.actor_service"     { class ActorService }
  package "app.services.diagnostico_service" { class DiagnosticoService }
  package "app.services.plan_service"      { class PlanService }
  package "app.services.export_service"    { class ExportService }
  package "app.services.extras_service"   { class ExtrasService }
}

package "app.routers" <<module>> {
  package "app.routers.auth"      { class AuthRouter }
  package "app.routers.users"     { class UsersRouter }
  package "app.routers.nna"       { class NNARouter }
  package "app.routers.catalogo"  { class CatalogoRouter }
  package "app.routers.actores"   { class ActoresRouter }
  package "app.routers.diagnosticos" { class DiagnosticosRouter }
  package "app.routers.planes"    { class PlanesRouter }
  package "app.routers.reportes"   { class ReportesRouter }
}

package "app.validators" <<module>> {
  package "app.validators.mexican_ids" { class MexicanIDValidator }
}

package "app.security"      { class SecurityModule }
package "app.database"      { class DatabaseModule }
package "app.dependencies" { class DependenciesModule }

package "app.config"      { class Settings }
package "app.main"        { class Application }
}

' Dependencias entre paquetes
"app.routers"    ..> "app.services"    : usa
"app.routers"    ..> "app.schemas"    : usa
"app.services"   ..> "app.models"      : usa
"app.services"   ..> "app.database"    : Session
"app.routers"    ..> "app.dependencies": Depends
"app.dependencies" ..> "app.security": verifica JWT
"app.security"   ..> "app.models.user": carga User
"app.main"       ..> "app.routers"     : include_router
"app.main"       ..> "app.models"      : create_all

@enduml
```

3.4.2 Descripción de Paquetes Backend

Paquete	Responsabilidad
app.models	Definición de las clases SQLAlchemy. Mapeo objeto-relacional. Sin lógica de negocio.
app.schemas	Clases Pydantic para validación de entrada (Create/Update) y serialización de salida (Response).

Paquete	Responsabilidad
app.services	Implementación de la lógica de negocio. Operaciones CRUD y reglas de dominio.
app.routers	Definición de endpoints HTTP. Reciben peticiones, validan con schemas y delegan a servicios.
app.validators	Validadores de datos específicos del dominio mexicano (RFC, CURP).
app.security	Funciones de hash/verify de contraseñas y creación/verificación de JWT.
app.database	Configuración del engine SQLAlchemy, SessionFactory y función get_db .
app.dependencies	Funciones de inyección de dependencias reutilizables: get_current_user , require_role .
app.config	Clase Settings que lee variables de entorno desde .env con pydantic-settings.

3.4.3 Estructura de Paquetes Frontend

```
@startuml Xolix_Paquetes_Frontend

skinparam packageStyle rectangle

title Diagrama de Paquetes – Frontend Xolix 3.0

package "src" <<directory>> {

    package "src.pages" <<directory>> {
        package "src.pages.nna" <<directory>> {
            class NnaDashboard
            class NuevoCasoNNA
            class CasoNNADetalle
            class EntrevistaWizard
            class FamiliogramaEditor
            class PersonasFamiliaresPage
            class RelacionesFamiliaresPage
            class DiagnosticoPage
            class PlanesPage
            class ResumenCasoNNAPage
        }
        class LoginPage
        class RegistroPage
        class Dashboard
        class ActoresList
        class ActorDetalle
        class ActorForm
        class ReportesPage
    }

    package "src.components" <<directory>> {
        class Topbar
        class ProtectedRoute
        class StatusBadge
        class ConfirmModal
    }

    package "src.context" <<directory>> {
```



```

+telefono: String[20]
+correo: String[100]
+direccion: String[300]
+ocupacion: String[150]
+documento_identificacion: String[50]
+numero_documento: String[100]
}

class DatosMedicosNNA <<entity>> {
  +id: Integer {PK}
  +caso_id: Integer {FK → nna_casos}
  +historial_medico: Text
  +alergias: Text
  +discapacidades: Text
  +tipo_sangre: String[5]
  +medico_responsable: String[200]
  +institucion_medica: String[200]
  +cartilla_vacunacion: JSON
}

class PersonaFamiliar <<entity>> {
  +id: Integer {PK}
  +caso_id: Integer {FK → nna_casos}
  +nombre: String[200]
  +edad: Integer
  +genero: GeneroNNA
  +rol_en_familia: String[100]
  +tipo_simbolo: TipoSimboloFamiliar
  +observaciones: Text
  +telefono: String[20]
  +direccion: String[300]
  +ocupacion: String[150]
  +escolaridad: String[100]
  +estado_salud: String[200]
  +vive_con_nna: Boolean
  +es_responsable_legal: Boolean
}

enum TipoSimboloFamiliar <<enumeration>> {
  normal
  clave
  cuidador
  agresor
  fallecido
}

class RelacionFamiliar <<entity>> {
  +id: Integer {PK}
  +caso_id: Integer {FK → nna_casos}
  +persona_origen_id: Integer {FK → nna_personas}
  +persona_destino_id: Integer {FK → nna_personas}
  +tipo_relacion: TipoRelacionFamiliar
  +descripcion: String[300]
  +bidireccional: Boolean
}

enum TipoRelacionFamiliar <<enumeration>> {
  biologica
  adoptiva
  acogimiento
  tutela
  conflictiva
  distante
}

```


[illegible]

3.5.2 Clases Boundary (Interfaz con el Usuario)

Las clases Boundary representan los puntos de contacto entre los actores externos y el sistema:

```
@startuml Xolix_ClassesBoundary
skinparam classBackgroundColor #E8F8F5
skinparam classBorderColor #1ABC9C

title Clases Boundary - Xolix 3.0
```

```

class IULogin <<boundary>> {
    -formulario_login: Form
    --
    +renderFormulario(): void
    +capturarCredenciales(): {correo, password}
    +mostrarError(mensaje: String): void
    +redirigirDashboard(): void
}

class IUExpediente <<boundary>> {
    -formulario_caso: Form
    -lista_casos: Table
    --
    +renderFormularioCreacion(): void
    +renderListaCasos(casos: List): void
    +renderDetalleCaso(caso: CasoNNAResponse): void
    +capturarDatosNNA(): CasoNNACreate
    +capturarDatosTutor(): TutorCreate
    +capturarDatosMedicos(): DatosMedicosCreate
    +mostrarError(campo: String, msg: String): void
}

class IUFamiliograma <<boundary>> {
    -canvas_reactflow: ReactFlowCanvas
    -panel_personas: Panel
    -panel_relaciones: Panel
    --
    +renderCanvas(grafo: JSON): void
    +agregarNodo(persona: PersonaFamiliar): void
    +agregarArista(relacion: RelacionFamiliar): void
    +exportarImagen(): Blob
    +capturarEstadoGrafo(): JSON
}

class IUDiagnostico <<boundary>> {
    -selector_tipo: Select
    -lista_indicadores: CheckboxList
    -campo_observaciones: TextArea
    --
    +renderTiposDiagnostico(): void
    +renderIndicadoresPorDerecho(indicadores: List): void
    +capturarEvaluacion(): DiagnosticoCreate
    +mostrarDerechosVulnerados(derechos: List): void
    +renderHistorial(diagnosticos: List): void
}

class IUPlan <<boundary>> {
    -formulario_plan: Form
    -lista_medidas: List
    -panel_seguimiento: Panel
    --
    +renderFormularioPlan(): void
    +renderMedidas(medidas: List): void

    +renderSeguimientos(seguimientos: List): void
    +capturarPlan(): PlanCreate
    +capturarSeguimiento(medida_id: Integer): SeguimientoCreate
    +mostrarBarraAvance(porcentaje: Integer): void
}

class IUActor <<boundary>> {
    -formulario_actor: Form
    -filtros_busqueda: FilterPanel

```

```

-lista_actores: Table
--
+renderListaActores(actores: List): void
+renderDetalleActor(actor: ActorResponse): void
+renderFiltros(): void
+capturarFiltros(): {municipio, tipo, derecho_id}
+capturarActor(): ActorCreate
}

class IUReportes <<boundary>> {
  -panel_kpi: Dashboard
  -grafica_derechos: BarChart
  -grafica_evolucion: LineChart
  --
  +renderKPIs(indicadores: Dict): void
  +renderGraficaDerechos(datos: List): void
  +renderGraficaEvolucion(datos: List): void
  +solicitarExportacionPDF(): void
  +solicitarExportacionExcel(): void
}

@enduml

```

3.5.3 Clases Control (Lógica del Caso de Uso)

```

@startuml Xolix_ClasesControl

skinparam classBackgroundColor #FEF9E7
skinparam classBorderColor #F39C12

title Clases Control – Xolix 3.0

class AccessCtr <<control>> {
  --
  +autenticar(correo: String, password: String, db: Session): TokenResponse
  +verificar_token(token: String): UserPayload
  +obtener_usuario_actual(token: String, db: Session): User
  +verificar_rol(usuario: User, roles: List[String]): bool
}

class ExpedienteCtr <<control>> {
  --
  +crear_caso(datos: CasonNACreate, creador_id: int, db: Session): CasonNA
  +obtener_caso(caso_id: int, db: Session): CasonNA
  +listar_casos(filtros: dict, db: Session): List[CasonNA]
  +actualizar_caso(caso_id: int, datos: CasonNAUpdate, db: Session): CasonNA
  +upsert_tutor(caso_id: int, datos: TutorCreate, db: Session): TutorNNA
  +upsert_datos_medicos(caso_id: int, datos: DatosMedicosCreate, db: Session): DatosMedicosNNA
}

class FamiliogramaCtr <<control>> {
  --
  +agregar_persona(caso_id: int, datos: PersonaCreate, db: Session): PersonaFamiliar
  +listar_personas(caso_id: int, db: Session): List[PersonaFamiliar]
  +agregar_relacion(caso_id: int, datos: RelacionCreate, db: Session): RelacionFamiliar
  +guardar_familiograma(caso_id: int, grafo_json: dict, db: Session): Familiograma
  +obtener_familiograma(caso_id: int, db: Session): Familiograma
}

class DiagnosticoCtr <<control>> {
  --
  +crear_diagnostico(datos: DiagnosticoCreate, responsable_id: int, db: Session): Diagnostico
  +evaluar_indicadores(diag_id: int, evaluaciones: List, db: Session): List[IndicadorDiagnostico]
}

```

```

+generar_derechos_vulnerados(diag_id: int, db: Session): List[DerechoVulnerado]
+listar_diagnosticos(caso_id: int, db: Session): List[Diagnostico]
+resumen_derechos_vulnerados(caso_id: int, db: Session): dict
}

class ActorCtr <<control>> {
  --
  +crear_actor(datos: ActorCreate, db: Session): Actor
  +buscar_actores(municipio: str, derecho_id: int, tipo: str, db: Session): List[Actor]
  +obtener_actor(actor_id: int, db: Session): Actor
  +agregar_servicio(actor_id: int, datos: ServicioCreate, db: Session): ServicioActor
  +listar_actores(filtros: dict, db: Session): List[Actor]
}

class PlanCtr <<control>> {
  --
  +crear_plan(datos: PlanCreate, responsable_id: int, db: Session): PlanRestitucion
  +agregar_medida(plan_id: int, datos: MedidaCreate, db: Session): MedidaRestitucion
  +registrar_seguimiento(medida_id: int, datos: SeguimientoCreate, registrador_id: int, db: Session): SeguimientoMedida
  +actualizar_avance_medida(medida_id: int, porcentaje: int, db: Session): MedidaRestitucion
  +listar_planes(caso_id: int, db: Session): List[PlanRestitucion]
}

class ReporteCtr <<control>> {
  --
  +calcular_indicadores_globales(db: Session): dict
  +frecuencia_derechos_vulnerados(db: Session): List[dict]
  +evolucion_casos(meses: int, db: Session): List[dict]
  +generar_pdf_casos(db: Session): StreamingResponse
  +generar_excel_casos(db: Session): StreamingResponse
  +generar_excel_actores(db: Session): StreamingResponse
}

class AuditCtr <<control>> {
  --
  +registrar(usuario_id: int, accion: str, entidad: str, entidad_id: int, detalles: dict, db: Session): AuditLog
  +listar_auditoria(db: Session, filtros: dict): List[AuditLog]
}

@enduml

```



DISEÑO DINÁMICO

9 Diagramas de Secuencia UML · Casos de Uso CU-01 a CU-09

4. DISEÑO DINÁMICO

El diseño dinámico describe el comportamiento del sistema a través del tiempo. Para cada caso de uso se presenta: descripción, actores involucrados, precondiciones, flujo principal y alternativo, postcondiciones, y el diagrama de secuencia UML 2.x con notación BCE.

Los participantes de cada diagrama siguen el patrón:

- **Actor** → usuario del sistema (figura de palo)
 - **Boundary** → interfaz de usuario (React) o endpoint de la API
 - **Control** → capa de servicio (service.py)
 - **Entity** → modelo ORM (SQLAlchemy)
 - **PostgreSQL** → base de datos relacional
-

4.1 Caso de Uso: Login

Descripción

Permite a un empleado de la fundación autenticarse en el sistema mediante correo electrónico y contraseña.

Actores

- **Actor primario:** Empleado (cualquier rol)

Precondiciones

- El usuario debe tener una cuenta registrada en el sistema.
- La cuenta debe estar activa (**activo = true**).

Flujo Principal

- El usuario ingresa al sistema y visualiza el formulario de inicio de sesión.
- El usuario escribe su correo electrónico.
- El usuario escribe su contraseña.
- El usuario presiona el botón "Iniciar sesión".
- El sistema valida que el correo y la contraseña no estén vacíos.
- El sistema busca al usuario en la base de datos por correo.
- El sistema verifica que la contraseña coincida con el hash almacenado.
- El sistema genera un JWT con el ID y rol del usuario.
- El sistema retorna el token al frontend.
- El frontend almacena el token en localStorage.
- El sistema redirige al Dashboard.

Flujos Alternativos

FA-01: Correo no encontrado

- Paso 6a: No se encuentra ningún usuario con ese correo.
- El sistema retorna HTTP 401 con mensaje "Credenciales inválidas".

- El frontend muestra el error en el formulario.
- El flujo termina.

FA-02: Contraseña incorrecta

- Paso 7a: La contraseña no coincide con el hash almacenado.
- El sistema retorna HTTP 401 con el mismo mensaje genérico "Credenciales inválidas".
- El flujo termina.

FA-03: Usuario inactivo

- Paso 7b: El usuario existe pero **activo = false**.
- El sistema retorna HTTP 403 con mensaje "Cuenta desactivada. Contacte al administrador."
- El flujo termina.

Postcondiciones

- El token JWT queda almacenado en **localStorage** del navegador.
- El usuario es redirigido al Dashboard.
- Si la autenticación falla, el formulario muestra el error correspondiente.

Diagrama de Secuencia UML

```
@startuml CU_Login

skinparam sequenceArrowThickness 2
skinparam sequenceGroupBorderThickness 2
skinparam backgroundColor #FAFAFA

title Diagrama de Secuencia – CU-01: Login\nPatrón BCE

actor "Empleado" as U
boundary "IULogin\n[:LoginPage.jsx]" as B
control "AccessCtr\n[:auth.py + security.py]" as C
entity "ORMUsuario\n[:models/user.py]" as E
database "PostgreSQL\n[:proyecto_escom]" as DB

activate U

U -> B : 1. accede a la aplicación
activate B

B --> U : 1.1 render(formulario_login)

U -> B : 2. escribe correo electrónico
U -> B : 3. escribe contraseña
U -> B : 4. presiona "Iniciar sesión"

B -> B : 4.1 validar campos no vacíos

alt [campos vacíos]
    B --> U : 4.1.1 mostrar error de validación local
else [campos completos]
    B -> C : 5. POST /api/auth/login\n{correo, password}
    activate C

    C -> C : 5.1 preparar credenciales

    C -> E : 5.2 obtener_por_correo(correo)
    activate E
```

```

E -> DB : 5.2.1 SELECT * FROM users\nWHERE correo = :correo
activate DB
DB --> E : 5.2.2 Row | null
deactivate DB

E --> C : 5.3 usuario | None
deactivate E

alt [usuario == None]
  C --> B : 5.4.1 HTTP 401 {detail: "Credenciales inválidas"}
  B --> U : 5.4.2 mostrar error "Credenciales inválidas"
else [usuario encontrado]
  C -> C : 5.5 verify_password(password, usuario.password)

  alt [password incorrecto]
    C --> B : 5.5.1 HTTP 401 {detail: "Credenciales inválidas"}
    B --> U : 5.5.2 mostrar error "Credenciales inválidas"
  else [password correcto]
    C -> C : 5.6 verificar activo(usuario.activo)

    alt [usuario inactivo]
      C --> B : 5.6.1 HTTP 403 {detail: "Cuenta desactivada"}
      B --> U : 5.6.2 mostrar error "Cuenta desactivada"
    else [usuario activo]
      C -> C : 5.7 create_access_token\n({user_id, rol, exp})

      opt [auditoria habilitada]
        C -> DB : 5.8 INSERT INTO audit_logs\n(usuario_id, accion="login", ...)
      end

      C --> B : 5.9 HTTP 200\n{access_token, token_type: "bearer"}
      deactivate C

      B -> B : 5.10 localStorage.setItem("token", access_token)
      B -> B : 5.11 decodificar payload JWT
      B --> U : 5.12 navigate("/dashboard")
    end
  end
end
end
end

deactivate B
deactivate U

@enduml

```

4.2 Caso de Uso: Crear Expediente

Descripción

Permite a un empleado con rol de psicólogo, trabajador social o coordinador registrar un nuevo caso NNA en el sistema.

Actores

- **Actor primario:** Psicólogo / Trabajador Social / Coordinador

Precondiciones

- El empleado debe estar autenticado con un rol autorizado.
- El NNA no debe tener ya un expediente abierto en el sistema.

Flujo Principal

- El usuario accede al módulo NNA y selecciona "Nuevo Caso".
- El sistema presenta el formulario de creación.
- El usuario captura los datos del NNA (nombre, fecha de nacimiento, género, nacionalidad).
- El usuario captura los datos del tutor/responsable legal.
- El usuario captura los datos médicos básicos.
- El usuario presiona "Guardar".
- El sistema valida todos los campos requeridos.
- El sistema verifica que no exista duplicado por CURP.
- El sistema crea el caso NNA.
- El sistema crea el registro del tutor vinculado al caso.
- El sistema crea el registro de datos médicos vinculado al caso.
- El sistema registra la acción en auditoría.
- El sistema redirige al detalle del nuevo caso.

Flujos Alternativos

FA-01: CURP duplicada

- Paso 8a: Ya existe un caso con la misma CURP.
- El sistema retorna HTTP 409 "Ya existe un caso registrado con esta CURP."

FA-02: Datos inválidos

- Paso 7a: Algún campo requerido está vacío o con formato incorrecto.
- Pydantic retorna HTTP 422 con la lista de errores de validación.

Postcondiciones

- El nuevo caso NNA queda registrado en **nna_casos**.
- El tutor queda registrado en **nna_tutores**.
- Los datos médicos quedan registrados en **nna_datos_medicos**.
- La acción queda registrada en **audit_logs**.

Diagrama de Secuencia UML

```
@startuml CU_CrearExpediente

title Diagrama de Secuencia – CU-02: Crear Expediente\nPatrón BCE

actor "Coordinador" as U
boundary "IUExpediente\n[:NuevoCasoNNA.jsx]" as B
control "ExpedienteCtr\n[:nna_service.py]" as C
entity "ORMCasoNNA\n[:models/nna.py]" as E
entity "ORMTutorNNA\n[:models/nna.py]" as ET
entity "ORMDatosMedicos\n[:models/nna.py]" as ED
database "PostgreSQL\n[:proyecto_escom]" as DB
```

```

activate U

U -> B : 1. navega a /nna/nuevo
activate B
B --> U : 1.1 render(FormularioNuevoCaso)

U -> B : 2. captura datos del NNA
U -> B : 3. captura datos del tutor
U -> B : 4. captura datos médicos
U -> B : 5. presiona "Guardar"

B -> B : 5.1 validar formulario (campos requeridos)

alt [validación local falla]
  B --> U : 5.1.1 mostrar errores de campo
else [formulario válido]

  B -> C : 6. POST /api/nna/casos\n{nna_nombre, nna_curp, nna_edad,\nna_genero, tutor, datos_medicos}
  activate C

  C -> C : 6.1 verificar rol (psicologo | trabajador_social | coordinador)

  alt [rol no autorizado]
    C --> B : 6.1.1 HTTP 403 Forbidden
    B --> U : 6.1.2 mostrar "Sin permisos"
  else [rol autorizado]

    C -> E : 6.2 verificar_curp_duplicada(nna_curp)
    activate E
    E -> DB : 6.2.1 SELECT id FROM nna_casos\nWHERE nna_curp = :curp
    activate DB
    DB --> E : 6.2.2 id | null
    deactivate DB
    E --> C : 6.3 caso_existente | None
    deactivate E

    alt [caso_existente != None]
      C --> B : 6.3.1 HTTP 409 "CURP duplicada"
      B --> U : 6.3.2 mostrar error de duplicado
    else [sin duplicado]

      C -> E : 6.4 crear_caso(datos, creador_id)
      activate E
      E -> DB : 6.4.1 INSERT INTO nna_casos\n(nna_nombre, nna_curp, nna_edad,\nna_genero, estado, creador_id)
      activate DB
      DB --> E : 6.4.2 caso.id (nuevo)
      deactivate DB
      E --> C : 6.5 caso: CasoNNA

      deactivate E

      C -> ET : 6.6 crear_tutor(caso_id, datos_tutor)
      activate ET
      ET -> DB : 6.6.1 INSERT INTO nna_tutores\n(caso_id, nombre, parentesco, ...)
      activate DB
      DB --> ET : 6.6.2 tutor.id
      deactivate DB
      ET --> C : 6.7 tutor: TutorNNA
      deactivate ET

      C -> ED : 6.8 crear_datos_medicos(caso_id, datos_medicos)
      activate ED
      ED -> DB : 6.8.1 INSERT INTO nna_datos_medicos\n(caso_id, historial, alergias, ...)

```

```

    activate DB
    DB --> ED : 6.8.2 datos.id
    deactivate DB
    ED --> C : 6.9 datos: DatosMedicosNNA
    deactivate ED

    C -> DB : 6.10 INSERT INTO audit_logs\n(usuario_id, accion="crear_caso",\nentidad="nna_casos", entidad_id=caso.id)
    activate DB
    DB --> C : 6.10.1 ok
    deactivate DB

    C -> DB : 6.11 COMMIT
    activate DB
    DB --> C : 6.11.1 ok
    deactivate DB

    C --> B : 6.12 HTTP 201\nCasoNNAResponse {id, nna_nombre, estado, ...}
    deactivate C

    B -> B : 6.13 navigate("/nna/casos/" + caso.id)
    B --> U : 6.14 mostrar mensaje "Caso creado exitosamente"
end
end
end

deactivate B
deactivate U

@enduml

```

4.3 Caso de Uso: Editar Expediente

Descripción

Permite actualizar los datos de un caso NNA existente.

Actores

- **Actor primario:** Psicólogo / Trabajador Social / Coordinador / Director

Precondiciones

- El caso debe existir en el sistema.
- El usuario debe estar autenticado con rol autorizado.

Flujo Principal

- El usuario accede al detalle del caso y selecciona "Editar".
- El sistema carga los datos actuales del caso en el formulario.
- El usuario modifica los campos deseados.
- El usuario presiona "Actualizar".
- El sistema valida los cambios.
- El sistema actualiza el registro en la base de datos.
- El sistema registra la acción en auditoría.

- El sistema retorna los datos actualizados.

Flujos Alternativos

FA-01: Caso no encontrado

- El sistema retorna HTTP 404 "Caso no encontrado."

FA-02: Sin cambios

- El sistema aplica la actualización de todas formas (upsert idempotente).

Postcondiciones

- Los datos del caso quedan actualizados en **nna_casos**.

Diagrama de Secuencia UML

```
@startuml CU_EditarExpediente

title Diagrama de Secuencia – CU-03: Editar Expediente\nPatrón BCE

actor "Trabajador Social" as U
boundary "IUExpediente\n[:CasoNNADetalle.jsx]" as B
control "ExpedienteCtr\n[:nna_service.py]" as C
entity "ORMCasoNNA\n[:models/nna.py]" as E
database "PostgreSQL\n[:proyecto_escom]" as DB

activate U

U -> B : 1. navega a /nna/casos/:id
activate B

B -> C : 1.1 GET /api/nna/casos/:id
activate C
C -> E : 1.2 obtener_caso(caso_id)
activate E
E -> DB : 1.2.1 SELECT * FROM nna_casos\nWHERE id = :caso_id
activate DB
DB --> E : 1.2.2 caso_data | null
deactivate DB
E --> C : 
deactivate E
C --> B : 
deactivate C

alt [caso not found]
    E --> C : 1.2.3 None
    C --> B : 1.3 HTTP 404 "Caso no encontrado"
    B --> U : 1.4 mostrar error 404
else [caso encontrado]
    E --> C : 1.2.4 caso: CasoNNA
    deactivate E
    C --> B : 1.3 HTTP 200 CasoNNAResponse
    deactivate C

    B --> U : 1.4 render(FormularioEdicion, datos=caso)

    U -> B : 2. modifica campos del expediente
    U -> B : 3. presiona "Actualizar"

    B -> B : 3.1 construir payload de cambios

    B -> C : 4. PUT /api/nna/casos/:id\n{campos_modificados}
    activate C
```

```

C -> E : 4.1 obtener_caso(caso_id)
activate E
E -> DB : 4.1.1 SELECT * FROM nna_casos\nWHERE id = :caso_id
activate DB
DB --> E : 4.1.2 caso_actual
deactivate DB
E --> C : 4.2 caso_actual: CasoNNA
deactivate E

loop [para cada campo modificado]
  C -> C : 4.3 setattr(caso_actual, campo, nuevo_valor)
end

C -> E : 4.4 flush() / commit()
activate E
E -> DB : 4.4.1 UPDATE nna_casos SET\n campo1=:v1, campo2=:v2\nWHERE id = :caso_id
activate DB
DB --> E : 4.4.2 rowcount = 1
deactivate DB
E --> C : 4.5 caso_actualizado: CasoNNA
deactivate E

C -> DB : 4.6 INSERT INTO audit_logs\n(accion="actualizar_caso", entidad_id=caso_id)
activate DB
DB --> C : 4.6.1 ok
deactivate DB

C --> B : 4.7 HTTP 200 CasoNNAResponse
deactivate C

B --> U : 4.8 mostrar mensaje "Expediente actualizado"
B --> U : 4.9 refresh datos en pantalla
end

deactivate B
deactivate U

@enduml

```

4.4 Caso de Uso: Registrar Diagnóstico

Descripción

Permite a un psicólogo o trabajador social registrar un diagnóstico para un caso NNA, evaluando los indicadores de derechos.

Actores

- **Actor primario:** Psicólogo / Trabajador Social

Precondiciones

- El caso NNA debe existir y estar activo.
- Debe existir al menos un indicador en el catálogo.

Flujo Principal

- El usuario navega al módulo de Diagnóstico del caso.
- El sistema carga los tipos de diagnóstico disponibles.
- El usuario selecciona el tipo de diagnóstico.
- El sistema carga los indicadores del catálogo agrupados por derecho.
- El usuario evalúa cada indicador (sí/no/parcial) y anota observaciones.
- El usuario presiona "Guardar diagnóstico".
- El sistema crea el registro del diagnóstico.
- Para cada indicador marcado como vulnerado, el sistema registra el derecho correspondiente como vulnerado.
- El sistema registra la acción en auditoría.
- El sistema muestra el diagnóstico guardado con el resumen de derechos vulnerados.

Flujos Alternativos

FA-01: Sin indicadores en catálogo

- El sistema muestra advertencia "El catálogo de indicadores está vacío."

FA-02: Diagnóstico del mismo tipo ya existente

- El sistema permite registrar múltiples diagnósticos del mismo tipo.

Postcondiciones

- El diagnóstico queda en **diagnosticos**.
- Los registros de evaluación quedan en **indicadores_diagnostico**.
- Los derechos vulnerados quedan en **derechos_vulnerados**.

Diagrama de Secuencia UML

```
@startuml CU_RegistrarDiagnostico

title Diagrama de Secuencia – CU-04: Registrar Diagnóstico\nPatrón BCE

actor "Psicologa" as U
boundary "IUDiagnostico\n[:DiagnosticoPage.jsx]" as B
control "DiagnosticoCtr\n[:diagnostico_service.py]" as C
entity "ORMDiagnostico\n[:models/diagnostico.py]" as ED
entity "ORMIndicadorDiag\n[:models/diagnostico.py]" as EI
entity "ORMDerechoVuln\n[:models/diagnostico.py]" as EV
entity "ORMIndicador\n[:models/catalogo.py]" as EC
database "PostgreSQL\n[:proyecto_escom]" as DB

activate U

U -> B : 1. navega a /nna/casos/:id/diagnostico
activate B

B -> C : 1.1 GET /api/catalogo/indicadores
activate C
C -> EC : 1.1.1 listar_indicadores()
activate EC
EC -> DB : 1.1.2 SELECT i.*, d.nombre as derecho\nFROM indicadores i\nJOIN derechos d ON i.derecho_id = d.id\nWHERE i.activo = true
activate DB
DB --> EC : 1.1.3 List[IndicadorRow]
deactivate DB
```

```

EC --> C : 1.1.4 indicadores: List[Indicador]
deactivate EC
C --> B : 1.1.5 HTTP 200 List[IndicadorResponse]
deactivate C

B --> U : 1.2 render(SelectorTipoDiagnostico)
B --> U : 1.3 render(ListaIndicadoresPorDerecho)

U -> B : 2. selecciona tipo = "inicial"
U -> B : 3. escribe observaciones generales

loop [para cada indicador]
  U -> B : 4.N evalua indicador N\n(valor: "si" | "no" | "parcial", vulnerado: bool)
end

U -> B : 5. presiona "Guardar diagnóstico"

B -> C : 6. POST /api/diagnosticos\n{caso_nna_id, tipo, observaciones,\n evaluaciones: [{indicador_id, v
alor, vulnerado}}]
activate C

C -> ED : 6.1 crear_diagnostico(caso_nna_id, tipo, responsable_id)
activate ED
ED -> DB : 6.1.1 INSERT INTO diagnosticos\n(caso_nna_id, tipo, fecha, responsable_id, observaciones)
activate DB
DB --> ED : 6.1.2 diagnostico.id
deactivate DB
ED --> C : 6.2 diagnostico: Diagnostico
deactivate ED

loop [para cada evaluacion en evaluaciones]
  C -> EI : 6.3.N crear_indicador_evaluacion\n(diagnostico_id, indicador_id, valor, vulnerado)
  activate EI
  EI -> DB : 6.3.N.1 INSERT INTO indicadores_diagnostico\n(diagnostico_id, indicador_id, valor, vulnerad
o)
  activate DB

  DB --> EI : 6.3.N.2 ok
  deactivate DB
  EI --> C : 6.3.N.3 ok
  deactivate EI
end

C -> EC : 6.4 obtener_derechos_de_indicadores_vulnerados\n(indicadores_vulnerados)
activate EC
EC -> DB : 6.4.1 SELECT DISTINCT derecho_id\nFROM indicadores\nWHERE id IN (:ids_vulnerados)
activate DB
DB --> EC : 6.4.2 List[derecho_id]
deactivate DB
EC --> C : 6.5 derechos_vulnerados_ids: List[int]
deactivate EC

loop [para cada derecho_id en derechos_vulnerados_ids]
  C -> EV : 6.6.N crear_derecho_vulnerado\n(diagnostico_id, derecho_id, severidad, generado_auto=True)
  activate EV
  EV -> DB : 6.6.N.1 INSERT INTO derechos_vulnerados\n(diagnostico_id, derecho_id, severidad, generado_a
utomaticamente)
  activate DB
  DB --> EV : 6.6.N.2 ok
  deactivate DB
  EV --> C : 6.6.N.3 ok
  deactivate EV
end

```

```

C -> DB : 6.7 INSERT INTO audit_logs\n(accion="crear_diagnostico", entidad_id=diagnostico.id)
activate DB
DB --> C : 6.7.1 ok
deactivate DB

C -> DB : 6.8 COMMIT
activate DB
DB --> C : 6.8.1 ok
deactivate DB

C --> B : 6.9 HTTP 201 DiagnosticoResponse\n{id, tipo, derechos_vulnerados: [...]}
deactivate C

B --> U : 6.10 mostrar resumen de derechos vulnerados
B --> U : 6.11 render(HistorialDiagnosticos)

deactivate B
deactivate U

@enduml

```

4.5 Caso de Uso: Determinar Derechos Vulnerados

Descripción

Permite al equipo consultar el resumen consolidado de derechos vulnerados para un caso NNA, como base para la planeación de la restitución.

Actores

- **Actor primario:** Coordinador / Psicólogo / Trabajador Social

Precondiciones

- El caso debe tener al menos un diagnóstico con indicadores evaluados.

Flujo Principal

- El usuario accede al módulo de Diagnóstico del caso.
- El usuario selecciona "Ver derechos vulnerados".
- El sistema consulta todos los diagnósticos del caso.
- El sistema agrega los derechos vulnerados por derecho y severidad.
- El sistema retorna el resumen.
- El frontend muestra el panel con los derechos vulnerados agrupados.

Postcondiciones

- El usuario puede visualizar qué derechos están vulnerados con su severidad y recomendación.

Diagrama de Secuencia UML

```

@startuml CU_DeterminarDerechos

title Diagrama de Secuencia – CU-05: Determinar Derechos Vulnerados\nPatrón BCE

```



```

actor "Coordinador" as U
boundary "IUDiagnostico\n[:DiagnosticoPage.jsx]" as B
control "DiagnosticoCtr\n[:diagnostico_service.py]" as C
entity "ORMDerechoVuln\n[:models/diagnostico.py]" as EV
entity "ORMDerecho\n[:models/catalogo.py]" as ED
database "PostgreSQL\n[:proyecto_escom]" as DB

activate U

U -> B : 1. navega a /nna/casos/:id/diagnostico
activate B
B --> U : 1.1 render(PanelDiagnostico)

U -> B : 2. selecciona "Ver derechos vulnerados"

B -> C : 3. GET /api/diagnosticos/caso/:id/derechos-vulnerados
activate C

C -> EV : 3.1 listar_derechos_vulnerados(caso_id)
activate EV
EV -> DB : 3.1.1 SELECT dv.derecho_id, dv.severidad,\n dv.recomendacion, d.nombre, d.categoria,\n COUNT(
dv.id) as frecuencia\nFROM derechos_vulnerados dv\nJOIN diagnosticos diag ON dv.diagnostico_id = diag.id
\nJOIN derechos d ON dv.derecho_id = d.id\nWHERE diag.caso_nna_id = :caso_id\nGROUP BY dv.derecho_id, dv
.severidad, d.nombre, d.categoria,\n dv.recomendacion\nORDER BY dv.severidad DESC
activate DB
DB --> EV : 3.1.2 List[DerechoVulneradoRow]
deactivate DB
EV --> C : 3.2 derechos_vulnerados: List[DerechoVulnerado]
deactivate EV

loop [para cada derecho_vulnerado]
  C -> ED : 3.3.N obtener_derecho(derecho_id)
  activate ED
  ED -> DB : 3.3.N.1 SELECT * FROM derechos\nWHERE id = :derecho_id
  activate DB
  DB --> ED : 3.3.N.2 derecho_data
  deactivate DB
  ED --> C : 3.3.N.3 derecho: Derecho
  deactivate ED

  C -> C : 3.4.N enriquecer(derecho_vulnerado, derecho)
end

C -> C : 3.5 agrupar_por_severidad(derechos_enriquecidos)

C --> B : 3.6 HTTP 200\n{criticos: [...], graves: [...], moderados: [...], leves: [...]}
deactivate C

alt [sin derechos vulnerados]
  B --> U : 4.1 mostrar "No se han registrado derechos vulnerados para este caso"
else [con derechos vulnerados]
  B --> U : 4.2 render(PanelDerechosVulnerados)
  B --> U : 4.3 render(TarjetasPorSeveridad)
  B --> U : 4.4 render(Recomendaciones)
end

deactivate B
deactivate U

@enduml

```

4.6 Caso de Uso: Buscar Actores

Descripción

Permite localizar actores (organizaciones, personas) que pueden apoyar la restitución de un derecho específico, filtrando por municipio, tipo y derecho vulnerado.

Actores

- **Actor primario:** Trabajador Social / Coordinador

Precondiciones

- Debe existir al menos un actor en el catálogo.

Flujo Principal

- El usuario navega a la sección de Actores.
- El sistema muestra el listado completo de actores activos.
- El usuario aplica filtros (municipio, tipo, derecho vulnerado).
- El sistema ejecuta la búsqueda con los filtros aplicados.
- El sistema retorna la lista filtrada de actores.
- El usuario selecciona un actor para ver su detalle.
- El sistema muestra el detalle completo del actor con sus servicios, horarios y requisitos.

Postcondiciones

- El usuario puede visualizar los actores disponibles para atender el derecho vulnerado.

Diagrama de Secuencia UML

```
@startuml CU_BuscarActores

title Diagrama de Secuencia – CU-06: Buscar Actores\nPatrón BCE

actor "Trabajador Social" as U
boundary "IUActor\n[:ActoresList.jsx]" as B
control "ActorCtr\n[:actor_service.py]" as C
entity "ORMActor\n[:models/actor.py]" as E
entity "ORMServicio\n[:models/actor.py]" as ES
database "PostgreSQL\n[:proyecto_escom]" as DB

activate U

U -> B : 1. navega a /actores
activate B

B -> C : 1.1 GET /api/actores?activo=true
activate C
C -> E : 1.1.1 listar_actores({})
activate E
E -> DB : 1.1.2 SELECT * FROM actores\nWHERE activo = true\nORDER BY nombre
activate DB
DB --> E : 1.1.3 List[ActorRow]
deactivate DB
E --> C : 1.1.4 actores: List[Actor]
deactivate E
```

```

C --> B : 1.1.5 HTTP 200 List[ActorListResponse]
deactivate C

B --> U : 1.2 render(TablaActores, actores=lista)
B --> U : 1.3 render(PanelFiltros)

U -> B : 2. selecciona filtro municipio = "Cuauhtémoc"
U -> B : 3. selecciona filtro derecho = "Derecho a la salud"
U -> B : 4. presiona "Buscar"

B -> C : 5. GET /api/actores\n?municipio=Cuauhtémoc&derecho_id=1
activate C

C -> E : 5.1 listar_actores\n({municipio: "Cuauhtémoc", derecho_id: 1})
activate E
E -> DB : 5.1.1 SELECT DISTINCT a.*\nFROM actores a\nJOIN actores_servicios s ON s.actor_id = a.id\nWHERE
E a.activo = true\nAND a.municipio ILIKE '%Cuauhtémoc%\nAND s.derecho_id = 1\nORDER BY a.nombre
activate DB
DB --> E : 5.1.2 List[ActorRow]
deactivate DB
E --> C : 5.2 actores_filtrados: List[Actor]
deactivate E

opt [sin resultados]
  C --> B : 5.3 HTTP 200 []
  B --> U : 5.4 mostrar "No se encontraron actores con esos criterios"
end

C --> B : 5.5 HTTP 200 List[ActorListResponse]
deactivate C

B --> U : 5.6 render(TablaActores actualizada, n_resultados)

U -> B : 6. selecciona actor "DIF Ciudad de México"

B -> C : 7. GET /api/actores/:actor_id
activate C
C -> E : 7.1 obtener_actor(actor_id)
activate E
E -> DB : 7.1.1 SELECT * FROM actores\nWHERE id = :actor_id
activate DB
DB --> E : 7.1.2 actor_data
deactivate DB
E --> C : 7.2 actor: Actor
deactivate E

C -> ES : 7.3 obtener_servicios(actor_id)
activate ES
ES -> DB : 7.3.1 SELECT s.*, d.nombre as derecho,\n r.descripcion as requisito\nFROM actores_servicios s
\nLEFT JOIN derechos d ON s.derecho_id = d.id\nLEFT JOIN servicios_requisitos r ON r.servicio_id = s.id\n
nWHERE s.actor_id = :actor_id
activate DB
DB --> ES : 7.3.2 List[ServicioRow]
deactivate DB
ES --> C : 7.4 servicios: List[ServicioActor]
deactivate ES

C --> B : 7.5 HTTP 200 ActorResponse {datos, servicios, horarios, responsables}
deactivate C

B --> U : 7.6 render(PaginaDetalleActor)

deactivate B
deactivate U

```

4.7 Caso de Uso: Crear Plan de Restitución

Descripción

Permite al coordinador crear un plan formal de restitución de derechos para un caso NNA, definiendo el objetivo, los derechos afectados y las medidas a tomar.

Actores

- **Actor primario:** Coordinador / Director

Precondiciones

- El caso NNA debe tener al menos un derecho vulnerado documentado en un diagnóstico.
- El usuario debe tener rol **coordinador** o **director**.

Flujo Principal

- El usuario navega al módulo de Planes del caso.
- El sistema muestra el formulario de creación de plan.
- El usuario captura el objetivo del plan.
- El usuario selecciona los derechos afectados del listado de derechos vulnerados.
- El usuario define las medidas de restitución (tipo, descripción, responsable, actor, fecha límite).
- El usuario presiona "Crear Plan".
- El sistema crea el plan de restitución.
- El sistema crea cada medida vinculada al plan.
- El sistema registra la acción en auditoría.
- El sistema muestra el plan creado con sus medidas.

Postcondiciones

- El plan queda en **planes_restitucion** con estado **activo**.
- Las medidas quedan en **medidas_restitucion** con estado **pendiente**.

Diagrama de Secuencia UML

```
@startuml CU_CrearPlan

title Diagrama de Secuencia – CU-07: Crear Plan de Restitución\nPatrón BCE

actor "Coordinador" as U
boundary "IUPlan\n[:PlanesPage.jsx]" as B
control "PlanCtr\n[:plan_service.py]" as C
entity "ORMPlan\n[:models/plan.py]" as EP
entity "ORMMedida\n[:models/plan.py]" as EM
database "PostgreSQL\n[:proyecto_escom]" as DB

activate U
```

```

U -> B : 1. navega a /nna/casos/:id/planes
activate B

ref over B, C : CU-05 Determinar Derechos Vulnerados\n(cargar derechos vulnerados del caso)

B --> U : 1.1 render(FormularioPlan)
B --> U : 1.2 render(ListaDerechosVulnerados para seleccionar)

U -> B : 2. captura objetivo del plan
U -> B : 3. selecciona derechos afectados
U -> B : 4. define fecha de inicio y fecha límite

loop [el usuario agrega medidas]
  U -> B : 5.N captura medida N\n(tipo, descripcion, responsable_id, actor_id, fecha_limite)
  B -> B : 5.N.1 agregar medida al estado local
end

U -> B : 6. presiona "Crear Plan"

B -> B : 6.1 validar que hay al menos 1 medida

alt [sin medidas]
  B --> U : 6.1.1 mostrar "Agrega al menos una medida"
else [con medidas]

  B -> C : 7. POST /api/planes\n{caso_nna_id, objetivo, derechos_afectados,\n responsable_id, fecha_inicio, fecha_termino,\n medidas: [{tipo, descripcion, actor_id, ...}]}
  activate C

  C -> C : 7.1 verificar rol (coordinador | director)

  C -> EP : 7.2 crear_plan\n(caso_nna_id, objetivo, derechos_afectados,\n responsable_id, fecha_inicio, fecha_termino)
  activate EP
  EP -> DB : 7.2.1 INSERT INTO planes_restitucion\n(caso_nna_id, objetivo, derechos_afectados,\n responsable_id, fecha_inicio, fecha_termino,\n estado='activo')
  activate DB
  DB --> EP : 7.2.2 plan.id
  deactivate DB
  EP --> C : 7.3 plan: PlanRestitucion
  deactivate EP

  loop [para cada medida en medidas]
    C -> EM : 7.4.N crear_medida\n(plan_id, tipo, descripcion,\n responsable_id, actor_id, fecha_limite)
    activate EM
    EM -> DB : 7.4.N.1 INSERT INTO medidas_restitucion\n(plan_id, tipo, descripcion, responsable_id,\n actor_id, estado='pendiente', porcentaje_avance=0)
    activate DB
    DB --> EM : 7.4.N.2 medida.id
    deactivate DB
    EM --> C : 7.4.N.3 medida: MedidaRestitucion

    deactivate EM
  end

  C -> DB : 7.5 INSERT INTO audit_logs\n(accion="crear_plan", entidad_id=plan.id)
  activate DB
  DB --> C : 7.5.1 ok
  deactivate DB

  C -> DB : 7.6 COMMIT
  activate DB
  DB --> C : 7.6.1 ok
  deactivate DB

```

```

C --> B : 7.7 HTTP 201 PlanResponse\n{id, objetivo, estado, medidas: [...]}
deactivate C

B --> U : 7.8 render(DetallePlan con medidas)
B --> U : 7.9 mostrar "Plan creado correctamente"
end

deactivate B
deactivate U

@enduml

```

4.8 Caso de Uso: Registrar Seguimiento

Descripción

Permite registrar el avance real de una medida de restitución, actualizando su porcentaje de cumplimiento.

Actores

- **Actor primario:** Psicólogo / Trabajador Social / Coordinador (según la medida)

Precondiciones

- La medida debe existir y estar en estado **pendiente** o **en_proceso**.
- El usuario debe ser el responsable de la medida o tener rol de coordinador/director.

Flujo Principal

- El usuario accede al plan y selecciona la medida a actualizar.
- El usuario presiona "Registrar seguimiento".
- El sistema muestra el formulario de seguimiento.
- El usuario captura la descripción del avance, el porcentaje de cumplimiento y observaciones.
- El usuario presiona "Guardar seguimiento".
- El sistema crea el registro de seguimiento.
- El sistema actualiza el porcentaje de avance de la medida.
- Si el porcentaje es 100%, el sistema cambia el estado de la medida a **completada**.

Postcondiciones

- El seguimiento queda en **seguimientos_medida**.
- La medida actualiza su **porcentaje_avance** y eventualmente su **estado**.

Diagrama de Secuencia UML

```

@startuml CU_RegistrarSeguimiento

title Diagrama de Secuencia – CU-08: Registrar Seguimiento\nPatrón BCE

actor "Psicologa" as U
boundary "IUPlan\n[:PlanesPage.jsx]" as B
control "PlanCtr\n[:plan_service.py]" as C

```

```

entity "ORMSeguimiento\n[:models/plan.py]" as ES
entity "ORMMedida\n[:models/plan.py]" as EM
database "PostgreSQL\n[:proyecto_escom]" as DB

activate U

U -> B : 1. selecciona medida del plan
activate B

B -> C : 1.1 GET /api/planes/medidas/:medida_id
activate C
C -> EM : 1.1.1 obtener_medida(medida_id)
activate EM
EM -> DB : 1.1.2 SELECT * FROM medidas_restitucion\nWHERE id = :medida_id
activate DB
DB --> EM : 1.1.3 medida_data
deactivate DB
EM --> C : 1.1.4 medida: MedidaRestitucion
deactivate EM
C --> B : 1.1.5 HTTP 200 MedidaResponse
deactivate C

B --> U : 1.2 render(DetalleMedida, barra_avance)
U -> B : 2. presiona "Registrar seguimiento"
B --> U : 2.1 render(FormularioSeguimiento)

U -> B : 3. captura descripcion_avance
U -> B : 4. captura porcentaje_cumplimiento (0-100)
U -> B : 5. captura observaciones (opcional)
U -> B : 6. presiona "Guardar"

B -> B : 6.1 validar porcentaje entre 0 y 100

B -> C : 7. POST /api/planes/medidas/:medida_id/seguimientos\n{descripcion_avance, porcentaje_cumplimiento, observaciones}
activate C

C -> EM : 7.1 verificar_medida_activa(medida_id)
activate EM
EM -> DB : 7.1.1 SELECT estado FROM medidas_restitucion\nWHERE id = :medida_id
activate DB
DB --> EM : 7.1.2 estado
deactivate DB
EM --> C : 7.2 medida.estado
deactivate EM

alt [estado == "cancelada" o "completada"]
  C --> B : 7.2.1 HTTP 400 "No se puede actualizar una medida completada/cancelada"
  B --> U : 7.2.2 mostrar error
else [estado válido]

  C -> ES : 7.3 crear_seguimiento\n(medida_id, registrado_por_id, descripcion, porcentaje)
  activate ES
  ES -> DB : 7.3.1 INSERT INTO seguimientos_medida\n(medida_id, registrado_por_id,\n fecha_seguimiento, descripcion_avance,\n porcentaje_cumplimiento, observaciones)

  activate DB
  DB --> ES : 7.3.2 seguimiento.id
  deactivate DB
  ES --> C : 7.4 seguimiento: SeguimientoMedida
  deactivate ES

  C -> EM : 7.5 actualizar_avance\n(medida_id, nuevo_porcentaje)
  activate EM

```

```

EM -> DB : 7.5.1 UPDATE medidas_restitucion\nSET porcentaje_avance = :porcentaje,\n fecha_actualizacio
n = NOW()\nWHERE id = :medida_id
activate DB
DB --> EM : 7.5.2 ok
deactivate DB
EM --> C : 7.6 ok
deactivate EM

alt [porcentaje_cumplimiento == 100]
C -> EM : 7.7 actualizar_estado(medida_id, "completada")
activate EM
EM -> DB : 7.7.1 UPDATE medidas_restitucion\nSET estado = 'completada'\nWHERE id = :medida_id
activate DB
DB --> EM : 7.7.2 ok
deactivate DB
EM --> C : 7.7.3 ok
deactivate EM
end

C -> DB : 7.8 COMMIT
activate DB
DB --> C : 7.8.1 ok
deactivate DB

C --> B : 7.9 HTTP 201 SeguimientoResponse\n{seguimiento, medida_actualizada}
deactivate C

B --> U : 7.10 actualizar barra de avance (porcentaje)
B --> U : 7.11 mostrar "Seguimiento registrado"

opt [medida completada]
B --> U : 7.12 mostrar badge "Medida completada"
end
end

deactivate B
deactivate U

@enduml

```

4.9 Caso de Uso: Generar Familiograma

Descripción

Permite registrar los integrantes del entorno familiar del NNA, establecer relaciones entre ellos y generar la representación gráfica interactiva del familiograma.

Actores

- **Actor primario:** Psicólogo / Trabajador Social

Precondiciones

- El caso NNA debe existir y estar activo.

Flujo Principal

- El usuario navega al módulo de Familiograma del caso.

- El sistema carga las personas familiares registradas y el familiograma existente (si hay).
- El usuario agrega nuevas personas al entorno familiar.
- El sistema registra cada persona en la base de datos.
- El usuario establece relaciones entre personas.
- El sistema registra cada relación en la base de datos.
- El sistema actualiza automáticamente el canvas de ReactFlow.
- El usuario posiciona visualmente los nodos en el canvas.
- El usuario presiona "Guardar familiograma".
- El sistema guarda el estado JSON del grafo (posición de nodos, aristas visuales).
- El sistema retorna el familiograma guardado.

Postcondiciones

- Las personas quedan en **nna_personas**.
- Las relaciones quedan en **nna_relaciones_familiares**.
- El grafo visual queda en **nna_familiogramas** como JSON.

Diagrama de Secuencia UML

```
@startuml CU_GenerarFamiliograma

title Diagrama de Secuencia – CU-09: Generar Familiograma\nPatrón BCE

actor "Psicologa" as U
boundary "IUFamiliograma\n[:FamiliogramaEditor.jsx]" as B
control "FamiliogramaCtr\n[:nna_service.py]" as C
entity "ORMPersona\n[:models/nna.py]" as EP
entity "ORMRelacion\n[:models/nna.py]" as ER
entity "ORMFamiliograma\n[:models/nna.py]" as EF
database "PostgreSQL\n[:proyecto_escom]" as DB

activate U

U -> B : 1. navega a /nna/casos/:id/familiograma
activate B

B -> C : 1.1 GET /api/nna/casos/:id/personas
activate C
C -> EP : 1.1.1 listar_personas(caso_id)
activate EP
EP -> DB : 1.1.2 SELECT * FROM nna_personas\nWHERE caso_id = :caso_id
activate DB
DB --> EP : 1.1.3 List[PersonaRow]
deactivate DB
EP --> C : 1.1.4 personas: List[PersonaFamiliar]
deactivate EP
C --> B : 1.1.5 HTTP 200 List[PersonaResponse]
deactivate C

B -> C : 1.2 GET /api/nna/casos/:id/familiograma
activate C
C -> EF : 1.2.1 obtener_familiograma(caso_id)
activate EF
EF -> DB : 1.2.2 SELECT * FROM nna_familiogramas\nWHERE caso_id = :caso_id\nORDER BY version DESC LIMIT 1
activate DB
DB --> EF : 1.2.3 familiograma_data | null
deactivate DB
```

```

EF --> C : 1.2.4 familiograma | None
deactivate EF
C --> B : 1.2.5 HTTP 200 FamiliogramaResponse | 204
deactivate C

opt [familiograma existente]
  B -> B : 1.3 cargar_grafo_en_canvas(familiograma.grafo_json)
end

B --> U : 1.4 render(CanvasReactFlow, personas, relaciones)

loop [usuario agrega personas]
  U -> B : 2.N llena formulario persona\n(nombre, edad, genero, rol, tipo_simbolo)
  U -> B : 2.N.1 presiona "Agregar persona"

  B -> C : 3.N POST /api/nna/casos/:id/personas\n{nombre, edad, genero, rol_en_familia, tipo_simbolo}
  activate C
  C -> EP : 3.N.1 crear_persona(caso_id, datos)
  activate EP
  EP -> DB : 3.N.2 INSERT INTO nna_personas\n(caso_id, nombre, edad, genero,\n rol_en_familia, tipo_simb
olo)
  activate DB
  DB --> EP : 3.N.3 persona.id

  deactivate DB
  EP --> C : 3.N.4 persona: PersonaFamiliar
  deactivate EP
  C --> B : 3.N.5 HTTP 201 PersonaResponse
  deactivate C

  B -> B : 3.N.6 agregar_nodo_canvas(persona)
end

loop [usuario establece relaciones]
  U -> B : 4.N selecciona persona_origen y persona_destino
  U -> B : 4.N.1 selecciona tipo_relacion
  U -> B : 4.N.2 presiona "Crear relación"

  B -> C : 5.N POST /api/nna/casos/:id/relaciones\n{persona_origen_id, persona_destino_id, tipo_relacion
}
  activate C
  C -> ER : 5.N.1 crear_relacion(caso_id, datos)
  activate ER
  ER -> DB : 5.N.2 INSERT INTO nna_relaciones_familiares\n(caso_id, persona_origen_id, persona_destino_id,\n tipo_relacion)
  activate DB
  DB --> ER : 5.N.3 relacion.id
  deactivate DB
  ER --> C : 5.N.4 relacion: RelacionFamiliar
  deactivate ER
  C --> B : 5.N.5 HTTP 201 RelacionResponse
  deactivate C

  B -> B : 5.N.6 agregar_arista_canvas(relacion)
end

U -> B : 6. reposiciona nodos en el canvas
U -> B : 7. presiona "Guardar familiograma"

B -> B : 7.1 capturar_estado_grafo() → grafo_json

B -> C : 8. PUT /api/nna/casos/:id/familiograma\n{grafo_json: {nodes: [...], edges: [...]}}
activate C

```

```
C --> EF : 8.1 upsert_familiograma(caso_id, grafo_json)
activate EF
EF --> DB : 8.1.1 INSERT INTO nna_familiogramas\n(caso_id, grafo_json, version)\nON CONFLICT (caso_id)\nDO UPDATE SET grafo_json = :grafo_json,\nversion = version + 1
activate DB
DB --> EF : 8.1.2 familiograma.id, version
deactivate DB
EF --> C : 8.2 familiograma: Familiograma
deactivate EF

C --> B : 8.3 HTTP 200 FamiliogramaResponse
deactivate C

B --> U : 8.4 mostrar "Familiograma guardado (v" + version + ")"

deactivate B
deactivate U

@enduml
```



DISEÑO DE PERSISTENCIA

Modelo Relacional · Diccionario de Datos · 12 Consultas SQL · Índices

```

entity "nna_casos" as NC {
    *id : INTEGER <<PK>>
    --
    *nna_nombre : VARCHAR(200)
    nna_curp : VARCHAR(18)
    nna_fecha_nacimiento : DATE

    nna_edad : INTEGER
    nna_genero : ENUM(generonna)
    nna_nacionalidad : VARCHAR(100)
    nna_estado_civil : VARCHAR(50)
    *estado : ENUM(estadocasonna)
    *creador_id : INTEGER <<FK→users>>
    fecha_creacion : TIMESTAMP = NOW()
    fecha_actualizacion : TIMESTAMP = NOW()
}

entity "nna_tutores" as NT {
    *id : INTEGER <<PK>>
    --
    *caso_id : INTEGER <<FK→nna_casos>>
    *nombre : VARCHAR(100)
    apellido_paterno : VARCHAR(100)
    apellido_materno : VARCHAR(100)
    curp : VARCHAR(18)
    rfc : VARCHAR(13)
    *parentesco : VARCHAR(50)
    telefono : VARCHAR(20)
    correo : VARCHAR(100)
    direccion : VARCHAR(300)
    ocupacion : VARCHAR(150)
    documento_identificacion : VARCHAR(50)
    numero_documento : VARCHAR(100)
}

entity "nna_datos_medicos" as NDM {
    *id : INTEGER <<PK>>
    --
    *caso_id : INTEGER <<FK→nna_casos>>
    historial_medico : TEXT
    alergias : TEXT
    discapacidades : TEXT
    tipo_sangre : VARCHAR(5)
    medico_responsable : VARCHAR(200)
    institucion_medica : VARCHAR(200)
    cartilla_vacunacion : JSON
}

entity "nna_personas" as NP {
    *id : INTEGER <<PK>>
    --
    *caso_id : INTEGER <<FK→nna_casos>>
    *nombre : VARCHAR(200)
    edad : INTEGER
    genero : ENUM(generonna)
    rol_en_familia : VARCHAR(100)
    tipo_simbolo : ENUM(tiposimbolofamiliar)
    observaciones : TEXT
    telefono : VARCHAR(20)
    direccion : VARCHAR(300)
    ocupacion : VARCHAR(150)
    escolaridad : VARCHAR(100)
    estado_salud : VARCHAR(200)
    vive_con_nna : BOOLEAN = false
}

```

```
es_responsable_legal : BOOLEAN = false
fecha_creacion : TIMESTAMP = NOW()
}

entity "nna_relaciones_familiares" as NRF {
    *id : INTEGER <<PK>>
    --
    *caso_id : INTEGER <<FK→nna_casos>>
    *persona_origen_id : INTEGER <<FK→nna_personas>>
    *persona_destino_id : INTEGER <<FK→nna_personas>>
    *tipo_relacion : ENUM(tiporelacionfamiliar)
    descripcion : VARCHAR(300)
    bidireccional : BOOLEAN = true
}

entity "nna_familiogramas" as NF {
    *id : INTEGER <<PK>>
    --
    *caso_id : INTEGER <<FK→nna_casos>>
    grafo_json : JSON
    version : INTEGER = 1
    fecha_creacion : TIMESTAMP = NOW()
    fecha_actualizacion : TIMESTAMP = NOW()
}

entity "nna_entrevistas" as NE {
    *id : INTEGER <<PK>>
    --
    *caso_id : INTEGER <<FK→nna_casos>>
    grado_negacion : INTEGER
    completada : BOOLEAN = false
    observaciones_negacion : TEXT
    frases_comunicadas : JSON
    dia_comun : JSON
}

entity "nna_observaciones" as NO {
    *id : INTEGER <<PK>>
    --
    *caso_id : INTEGER <<FK→nna_casos>>
    persona_familiar_id : INTEGER <<FK→nna_personas>>
    tipo : VARCHAR(50)
    descripcion : TEXT
    fecha_observacion : DATE
}

' ████████████████████████████████████████████████████████████████████████████
' CATÁLOGO
' ████████████████████████████████████████████████████████████████████████████

entity "derechos" as D {
    *id : INTEGER <<PK>>
    --
    *nombre : VARCHAR(200)
    descripcion : TEXT
    categoria : ENUM(categoriaderecho)
    articulo_referencia : VARCHAR(100)
    activo : BOOLEAN = true
}

entity "indicadores" as IND {
    *id : INTEGER <<PK>>
    --
```

}

1

}

}

}

}


```

AS2 ||--o{ SR      : requiere

DG ||--o{ ID       : evalua
DG ||--o{ DV       : genera

IND ||--o{ ID      : evaluado en

PR ||--o{ MR       : contiene
MR ||--o{ SM       : registra

@enduml

```

5.2 Diccionario de Datos

Tabla: users

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	SÍ	-	Identificador único auto-incremental del usuario
nombre	VARCHAR	50	NO	-	-	Nombre de pila del empleado
apellido_paterno	VARCHAR	50	NO	-	-	Apellido paterno del empleado
apellido_materno	VARCHAR	50	NO	-	-	Apellido materno del empleado
rfc	VARCHAR	13	NO	-	-	RFC del empleado (UNIQUE, validado con regex mexicano)
curp	VARCHAR	18	NO	-	-	CURP del empleado (UNIQUE, 18 caracteres)
sexo	VARCHAR	10	NO	-	-	Sexo biológico del empleado: 'M', 'F'
fecha_nacimiento	DATE	-	NO	-	-	Fecha de nacimiento del empleado

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
edad	INTEGER	-	NO	-	-	Edad calculada del empleado
estado	VARCHAR	50	NO	-	-	Estado de la República donde vive
municipio	VARCHAR	100	NO	-	-	Municipio o alcaldía de residencia
colonia	VARCHAR	100	NO	-	-	Colonia de residencia
calle	VARCHAR	100	NO	-	-	Nombre de la calle
numero	VARCHAR	20	NO	-	-	Número exterior
codigo_postal	VARCHAR	5	NO	-	-	Código postal de 5 dígitos
calles_aledanas	TEXT	-	SÍ	-	-	Referencias de calles cercanas
tipo_personal	VARCHAR	20	NO	-	-	Tipo: 'empleado', 'voluntario'
rol	VARCHAR	30	NO	-	-	Rol en el sistema: director, coordinador, psicologo, trabajador_social, legal
correo	VARCHAR	100	NO	-	-	Correo electrónico (UNIQUE, usado para login)
password	VARCHAR	255	NO	-	-	Hash bcrypt de la contraseña
activo	BOOLEAN	-	SÍ	-	-	Estado de activación de la cuenta
verificado	BOOLEAN	-	SÍ	-	-	Indica si la cuenta fue verificada por el director
foto_perfil	VARCHAR	500	SÍ	-	-	Ruta o URL de la foto de perfil

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
fecha_creacion	TIMESTAMP	-	Sí	-	-	Fecha y hora de registro en el sistema

Tabla: nna_casos

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	Sí	-	Identificador único del caso NNA
nna_nombre	VARCHAR	200	NO	-	-	Nombre completo del NNA
nna_curp	VARCHAR	18	Sí	-	-	CURP del NNA
nna_fecha_nacimiento	DATE	-	Sí	-	-	Fecha de nacimiento del NNA
nna_edad	INTEGER	-	Sí	-	-	Edad del NNA al momento del registro
nna_genero	ENUM	-	Sí	-	-	Género: masculino, femenino, no_binario, pre_fiero_no_decir
nna_nacionalidad	VARCHAR	100	Sí	-	-	Nacionalidad del NNA
nna_estado_civil	VARCHAR	50	Sí	-	-	Estado civil del NNA (relevante para adolescentes)
estado	ENUM	-	NO	-	-	Estado del caso: activo, seguimiento, cerrado, archivado
creador_id	INTEGER	-	NO	-	users.id	Usuario que creó el caso
fecha_creacion	TIMESTAMP	-	Sí	-	-	Fecha de apertura del caso
fecha_actualizacion	TIMESTAMP	-	Sí	-	-	Última modificación del caso

Tabla: nna_tutores

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	SÍ	-	Identificador único del tutor
caso_id	INTEGER	-	NO	-	nna_casos.id	Caso al que pertenece el tutor
nombre	VARCHAR	100	NO	-	-	Nombre del tutor/responsable legal
apellido_paterno	VARCHAR	100	SÍ	-	-	Apellido paterno del tutor
apellido_materno	VARCHAR	100	SÍ	-	-	Apellido materno del tutor
curp	VARCHAR	18	SÍ	-	-	CURP del tutor
rfc	VARCHAR	13	SÍ	-	-	RFC del tutor
parentesco	VARCHAR	50	NO	-	-	Parentesco con el NNA: madre, padre, abuelo/a, tío/a, tutor legal
telefono	VARCHAR	20	SÍ	-	-	Teléfono de contacto del tutor
correo	VARCHAR	100	SÍ	-	-	Correo electrónico del tutor
direccion	VARCHAR	300	SÍ	-	-	Dirección de residencia del tutor
ocupacion	VARCHAR	150	SÍ	-	-	Ocupación laboral del tutor
documento_identificacion	VARCHAR	50	SÍ	-	-	Tipo de documento: INE, pasaporte, etc.
numero_documento	VARCHAR	100	SÍ	-	-	Número del documento de identificación

Tabla: nna_datos_medicos

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	SÍ	-	Identificador único
caso_id	INTEGER	-	NO	-	nna_casos.id	Caso al que pertenecen los datos médicos
historial_medico	TEXT	-	SÍ	-	-	Descripción del historial médico relevante
alergias	TEXT	-	SÍ	-	-	Alergias conocidas del NNA
discapacidades	TEXT	-	SÍ	-	-	Discapacidades documentadas
tipo_sangre	VARCHAR	5	SÍ	-	-	Tipo de sangre: A+, A-, B+, B-, AB+, AB-, O+, O-
medico_responsable	VARCHAR	200	SÍ	-	-	Nombre del médico tratante
institucion_medica	VARCHAR	200	SÍ	-	-	Institución médica de atención
cartilla_vacunacion	JSON	-	SÍ	-	-	Array de objetos: [{vacuna, fecha, dosis}]

Tabla: nna_personas

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	SÍ	-	Identificador único de la persona familiar
caso_id	INTEGER	-	NO	-	nna_casos.id	Caso al que pertenece
nombre	VARCHAR	200	NO	-	-	Nombre completo de la persona
edad	INTEGER	-	SÍ	-	-	Edad de la persona

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
genero	ENUM	-	Sí	-	-	Género: masculino, femenino, no_binario
rol_en_familia	VARCHAR	100	Sí	-	-	Rol: padre, madre, hermano/a, abuelo/a, tío/a, etc.
tipo_simbolo	ENUM	-	Sí	-	-	Símbolo en el familiograma: normal, clave, cuidador, agresor, fallecido
observaciones	TEXT	-	Sí	-	-	Notas clínicas sobre la persona
telefono	VARCHAR	20	Sí	-	-	Teléfono de contacto
direccion	VARCHAR	300	Sí	-	-	Dirección de residencia
ocupacion	VARCHAR	150	Sí	-	-	Ocupación laboral
escolaridad	VARCHAR	100	Sí	-	-	Nivel de escolaridad
estado_salud	VARCHAR	200	Sí	-	-	Estado de salud conocido
vive_con_nna	BOOLEAN	-	Sí	-	-	Indica si vive en el mismo hogar que el NNA
es_responsable_legal	BOOLEAN	-	Sí	-	-	Indica si es el responsable legal del NNA
fecha_creacion	TIMESTAMP	-	Sí	-	-	Fecha de registro

Tabla: nna_relaciones_familiares

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	Sí	-	Identificador único de la relación
caso_id	INTEGER	-	NO	-	nna_casos.id	Caso al que pertenece

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
persona_origen_id	INTEGER	-	NO	-	nna_personas.id	Persona de origen de la relación
persona_destino_id	INTEGER	-	NO	-	nna_personas.id	Persona de destino de la relación
tipo_relacion	ENUM	-	NO	-	-	Tipo: biológica, adoptiva, acogimiento, tutela, conflictiva, distante, apoyo
descripcion	VARCHAR	300	SÍ	-	-	Descripción adicional de la relación
bidireccional	BOOLEAN	-	SÍ	-	-	Si la arista del grafo es bidireccional

Tabla: nna_familiogramas

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	SÍ	-	Identificador único del familiograma
caso_id	INTEGER	-	NO	-	nna_casos.id	Caso al que pertenece
grafo_json	JSON	-	SÍ	-	-	Estado completo del grafo ReactFlow: {n odes:[{id,data, position}], edg es:[{id,source,t arget}]}
version	INTEGER	-	SÍ	-	-	Versión del familiograma (incrementa en cada guardado)
fecha_creacion	TIMESTAMP	-	SÍ	-	-	Fecha de creación
fecha_actualizacion	TIMESTAMP	-	SÍ	-	-	Última actualización

Tabla: derechos

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	SÍ	-	Identificador único del derecho
nombre	VARCHAR	200	NO	-	-	Nombre del derecho según catálogo
descripcion	TEXT	-	SÍ	-	-	Descripción del derecho
categoria	ENUM	-	SÍ	-	-	Categoría: salud, educacion, identidad, familia, proteccion, participacion, alimentacion, vivienda, otro
articulo_referencia	VARCHAR	100	SÍ	-	-	Artículo de ley o tratado internacional de referencia
activo	BOOLEAN	-	SÍ	-	-	Si el derecho está activo en el catálogo

Tabla: indicadores

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	SÍ	-	Identificador único del indicador
derecho_id	INTEGER	-	NO	-	derechos.id	Derecho al que pertenece este indicador
nombre	VARCHAR	300	NO	-	-	Enunciado del indicador (pregunta evaluable)
descripcion	TEXT	-	SÍ	-	-	Descripción detallada del indicador
tipo_evaluacion	VARCHAR	50	SÍ	-	-	Tipo de evaluación: si_no, escala, texto

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
activo	BOOLEAN	-	Sí	-	-	Si el indicador está activo

Tabla: actores

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	Sí	-	Identificador único del actor
nombre	VARCHAR	300	NO	-	-	Nombre del actor u organización
tipo	ENUM	-	NO	-	-	Tipo: gobierno, civil, empresa, persona_fisica
descripcion	TEXT	-	Sí	-	-	Descripción de la organización y sus actividades
direccion	VARCHAR	300	Sí	-	-	Dirección física
municipio	VARCHAR	100	Sí	-	-	Municipio o alcaldía de ubicación
estado	VARCHAR	100	Sí	-	-	Estado de la República
pais	VARCHAR	100	Sí	-	-	País (por defecto México)
telefono	VARCHAR	20	Sí	-	-	Teléfono principal de contacto
correo	VARCHAR	100	Sí	-	-	Correo electrónico institucional
sitio_web	VARCHAR	200	Sí	-	-	Sitio web oficial
redes_sociales	JSON	-	Sí	-	-	Redes sociales: {facebook, twitter, instagram}
activo	BOOLEAN	-	Sí	-	-	Si el actor está activo en el directorio

Tabla: actores_servicios

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	SÍ	-	Identificador único del servicio
actor_id	INTEGER	-	NO	-	actores.id	Actor que ofrece el servicio
derecho_id	INTEGER	-	SÍ	-	derechos.id	Derecho que atiende este servicio
nombre	VARCHAR	300	NO	-	-	Nombre del servicio
descripcion	TEXT	-	SÍ	-	-	Descripción del servicio
tipo	ENUM	-	NO	-	-	Tipo: servicio, producto
es_gratuito	BOOLEAN	-	SÍ	-	-	Si el servicio no tiene costo
costo	DECIMAL	10,2	SÍ	-	-	Costo en pesos mexicanos (si aplica)
disponibilidad	VARCHAR	100	SÍ	-	-	Descripción de disponibilidad
duracion_estimada	VARCHAR	100	SÍ	-	-	Duración estimada del servicio
activo	BOOLEAN	-	SÍ	-	-	Si el servicio está disponible

Tabla: diagnosticos

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	SÍ	-	Identificador único del diagnóstico
caso_nna_id	INTEGER	-	NO	-	nna_casos.id	Caso al que pertenece
tipo	ENUM	-	NO	-	-	Tipo: inicial, nna, tutor, entorno
fecha	DATE	-	NO	-	-	Fecha en que se realizó el diagnóstico

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
responsable_id	INTEGER	-	Sí	-	users.id	Usuario que realizó el diagnóstico
observaciones	TEXT	-	Sí	-	-	Observaciones narrativas del diagnóstico
completado	BOOLEAN	-	Sí	-	-	Si el diagnóstico fue completado formalmente
fecha_creacion	TIMESTAMP	-	Sí	-	-	Fecha de registro en el sistema

Tabla: indicadores_diagnostico

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	Sí	-	Identificador único de la evaluación
diagnostico_id	INTEGER	-	NO	-	diagnosticos.id	Diagnóstico al que pertenece
indicador_id	INTEGER	-	NO	-	indicadores.id	Indicador evaluado
valor	VARCHAR	20	Sí	-	-	Valor de la evaluación: 'si', 'no', 'parcial'
observacion	TEXT	-	Sí	-	-	Observación específica sobre el indicador
vulnerado	BOOLEAN	-	Sí	-	-	Si este indicador fue marcado como vulnerado

Tabla: derechos_vulnerados

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	Sí	-	Identificador único
diagnostico_id	INTEGER	-	NO	-	diagnosticos.id	Diagnóstico que generó este registro

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
derecho_id	INTEGER	-	NO	-	derechos.id	Derecho que fue vulnerado
severidad	ENUM	-	SÍ	-	-	Severidad: leve, moderada, grave, critica
recomendacion	TEXT	-	SÍ	-	-	Recomendación de acción para restituir el derecho
generado_automaticamente	BOOLEAN	-	SÍ	-	-	True si fue generado por el sistema; False si fue ingresado manualmente

Tabla: planes_restitucion

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	SÍ	-	Identificador único del plan
caso_nna_id	INTEGER	-	NO	-	nna_casos.id	Caso al que pertenece el plan
objetivo	TEXT	-	NO	-	-	Objetivo general del plan de restitución
derechos_afectados	JSON	-	SÍ	-	-	Array de IDs de derechos afectados: [1, 3, 5]
responsable_id	INTEGER	-	SÍ	-	users.id	Usuario coordinador del plan
fecha_inicio	DATE	-	SÍ	-	-	Fecha de inicio del plan
fecha_termino	DATE	-	SÍ	-	-	Fecha estimada de término
estado	ENUM	-	NO	-	-	Estado: borrador, activo, pausado, completado, cancelado

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
observaciones	TEXT	-	Sí	-	-	Notas adicionales sobre el plan
fecha_creacion	TIMESTAMP	-	Sí	-	-	Fecha de creación
fecha_actualizacion	TIMESTAMP	-	Sí	-	-	Última modificación

Tabla: medidas_restitucion

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	Sí	-	Identificador único de la medida
plan_id	INTEGER	-	NO	-	planes_restitucion.id	Plan al que pertenece
tipo	ENUM	-	NO	-	-	Tipo: psicológica, legal, médica, educativa, social, económica, otra
descripcion	TEXT	-	NO	-	-	Descripción de la medida a ejecutar
responsable_id	INTEGER	-	Sí	-	users.id	Usuario responsable de ejecutar la medida
actor_id	INTEGER	-	Sí	-	actores.id	Actor externo que apoya la medida
recursos_requeridos	TEXT	-	Sí	-	-	Recursos necesarios para ejecutar la medida
estado	ENUM	-	NO	-	-	Estado: pendiente, en_proceso, completada, cancelada
porcentaje_avance	INTEGER	-	Sí	-	-	Porcentaje de cumplimiento (0-100)

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
fecha_inicio	DATE	-	Sí	-	-	Fecha de inicio de la medida
fecha_limite	DATE	-	Sí	-	-	Fecha límite de cumplimiento
fecha_creacion	TIMESTAMP	-	Sí	-	-	Fecha de creación
fecha_actualizacion	TIMESTAMP	-	Sí	-	-	Última modificación

Tabla: seguimientos_medida

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	Sí	-	Identificador único del seguimiento
medida_id	INTEGER	-	NO	-	medidas_restitucion.id	Medida a la que pertenece el seguimiento
registrado_por_id	INTEGER	-	Sí	-	users.id	Usuario que registró el seguimiento
fecha_seguimiento	DATE	-	NO	-	-	Fecha en que se realizó el seguimiento
descripcion_avance	TEXT	-	NO	-	-	Descripción narrativa del avance
porcentaje_cumplimiento	INTEGER	-	Sí	-	-	Porcentaje alcanzado en este seguimiento (0-100)
observaciones	TEXT	-	Sí	-	-	Observaciones adicionales
evidencias	JSON	-	Sí	-	-	Lista de evidencias: [{nombre, archivo_path}]
fecha_creacion	TIMESTAMP	-	Sí	-	-	Fecha de registro en el sistema

Tabla: audit_logs

Campo	Tipo	Tamaño	Nulo	PK	FK	Descripción
id	INTEGER	-	NO	SÍ	-	Identificador único del registro de auditoría
usuario_id	INTEGER	-	SÍ	-	users.id	Usuario que realizó la acción
accion	VARCHAR	100	NO	-	-	Nombre de la acción: crear_caso, actualizar_caso, crear_diagnostico, etc.
entidad	VARCHAR	100	SÍ	-	-	Nombre de la tabla afectada
entidad_id	INTEGER	-	SÍ	-	-	ID del registro afectado
detalles	JSON	-	SÍ	-	-	Detalles adicionales de la acción en formato JSON
fecha	TIMESTAMP	-	SÍ	-	-	Fecha y hora de la acción

5.3 Diseño de Consultas

QueryLogin

Atributo	Detalle
Nombre	QueryLogin
Objetivo	Recuperar los datos de autenticación de un usuario a partir de su correo electrónico
Parámetros	correo VARCHAR(100)
Resultado	Fila única con id, password hash, rol y estado activo

[SQL]

```
-- QueryLogin
SELECT
  u.id,
  u.correo,
  u.password,
  u.rol,
  u.activo,
  u.nombre,
  u.apellido_paterno
FROM
```

```

    users u
WHERE
    u.correo = :correo
LIMIT 1;

```

Nota: Se busca por campo indexado UNIQUE. No requiere índice adicional. El resultado puede ser NULL si el correo no existe; en ese caso el servicio retorna HTTP 401.

QueryExpedientePorId

Atributo	Detalle
Nombre	QueryExpedientePorId
Objetivo	Obtener todos los datos de un caso NNA con su tutor y datos médicos
Parámetros	caso_id INTEGER
Resultado	Objeto complejo con caso + tutor + datos_medicos

[SQL]

```

-- QueryExpedientePorId
SELECT
    nc.id,
    nc.nna_nombre,
    nc.nna_curp,
    nc.nna_fecha_nacimiento,
    nc.nna_edad,
    nc.nna_genero,
    nc.nna_nacionalidad,
    nc.nna_estado_civil,
    nc.estado,
    nc.creador_id,
    nc.fecha_creacion,
    nc.fecha_actualizacion,
    -- Tutor
    nt.id AS tutor_id,
    nt.nombre AS tutor_nombre,
    nt.apellido_paterno AS tutor_apellido_paterno,
    nt.parentesco AS tutor_parentesco,
    nt.telefono AS tutor_telefono,
    nt.correo AS tutor_correo,
    -- Datos médicos
    ndm.id AS medicos_id,
    ndm.historial_medico,
    ndm.alergias,
    ndm.tipo_sangre,
    ndm.cartilla_vacunacion
FROM
    nna_casos nc
    LEFT JOIN nna_tutores nt ON nt.caso_id = nc.id
    LEFT JOIN nna_datos_medicos ndm ON ndm.caso_id = nc.id
WHERE
    nc.id = :caso_id;

```

QueryCrearExpediente

Atributo	Detalle
Nombre	QueryCrearExpediente
Objetivo	Insertar un nuevo caso NNA en la base de datos
Parámetros	nna_nombre, nna_curp, nna_fecha_nacimiento, nna_edad, nna_genero, nna_nacionalidad, estado, creador_id
Resultado	ID del nuevo caso creado

[SQL]

```
-- QueryCrearExpediente
INSERT INTO nna_casos (
    nna_nombre,
    nna_curp,
    nna_fecha_nacimiento,
    nna_edad,
    nna_genero,
    nna_nacionalidad,
    estado,
    creador_id
) VALUES (
    :nna_nombre,
    :nna_curp,
    :nna_fecha_nacimiento,
    :nna_edad,
    :nna_genero,
    :nna_nacionalidad,
    :estado,
    :creador_id
)
RETURNING id, fecha_creacion;
```

QueryActualizarExpediente

Atributo	Detalle
Nombre	QueryActualizarExpediente
Objetivo	Actualizar los campos modificados de un caso NNA existente
Parámetros	caso_id, campos dinámicos según lo que se modifique
Resultado	Número de filas afectadas (1)

[SQL]

```
-- QueryActualizarExpediente
UPDATE nna_casos
SET
    nna_nombre      = COALESCE(:nna_nombre, nna_nombre),
    nna_curp        = COALESCE(:nna_curp, nna_curp),
    nna_edad        = COALESCE(:nna_edad, nna_edad),
    nna_genero      = COALESCE(:nna_genero, nna_genero),
    nna_nacionalidad = COALESCE(:nna_nacionalidad, nna_nacionalidad),
    estado          = COALESCE(:estado, estado),
    fecha_actualizacion = NOW()
WHERE
    id = :caso_id
```

```
RETURNING id, fecha_actualizacion;
```

QueryDiagnosticosPorNNA

Atributo	Detalle
Nombre	QueryDiagnosticosPorNNA
Objetivo	Listar todos los diagnósticos registrados para un caso NNA, ordenados del más reciente al más antiguo
Parámetros	caso_id INTEGER
Resultado	Lista de diagnósticos con nombre del responsable

[SQL]

```
-- QueryDiagnosticosPorNNA
SELECT
  d.id,
  d.tipo,
  d.fecha,
  d.observaciones,
  d.completado,
  d.fecha_creacion,
  u.nombre      AS responsable_nombre,
  u.apellido_paterno AS responsable_apellido,
  u.rol         AS responsable_rol,
  COUNT(dv.id)  AS total_derechos_vulnerados
FROM
  diagnosticos d
  LEFT JOIN users u  ON u.id = d.responsable_id
  LEFT JOIN derechos_vulnerados dv ON dv.diagnostico_id = d.id
WHERE
  d.caso_nna_id = :caso_id
GROUP BY
  d.id, u.nombre, u.apellido_paterno, u.rol
ORDER BY
  d.fecha DESC,
  d.fecha_creacion DESC;
```

QueryDerechosVulnerados

Atributo	Detalle
Nombre	QueryDerechosVulnerados
Objetivo	Obtener el resumen consolidado de derechos vulnerados para un caso, agrupados por derecho y severidad máxima
Parámetros	caso_id INTEGER
Resultado	Lista de derechos con severidad máxima y recomendación más reciente

[SQL]

```
-- QueryDerechosVulnerados
SELECT
```

```

    der.id          AS derecho_id,
    der.nombre      AS derecho_nombre,
    der.categoria   AS derecho_categoria,
    der.articulo_referencia,
    MAX(dv.severidad::text) AS severidad_maxima,
    COUNT(dv.id)    AS frecuencia,
    (
        SELECT dv2.recomendacion
        FROM derechos_vulnerados dv2
        JOIN diagnosticos d2 ON d2.id = dv2.diagnostico_id
        WHERE dv2.derecho_id = der.id
          AND d2.caso_nna_id = :caso_id
        ORDER BY d2.fecha DESC
        LIMIT 1
    ) AS recomendacion_mas_reciente
FROM
    derechos_vulnerados dv
    JOIN diagnosticos diag ON diag.id = dv.diagnostico_id
    JOIN derechos der ON der.id = dv.derecho_id
WHERE
    diag.caso_nna_id = :caso_id
GROUP BY
    der.id, der.nombre, der.categoria, der.articulo_referencia
ORDER BY
    CASE MAX(dv.severidad::text)
        WHEN 'critica' THEN 1
        WHEN 'grave'   THEN 2
        WHEN 'moderada' THEN 3
        WHEN 'leve'    THEN 4
    END;

```

QueryActoresPorMunicipio

Atributo	Detalle
Nombre	QueryActoresPorMunicipio
Objetivo	Buscar actores activos en un municipio específico, opcionalmente filtrados por tipo
Parámetros	municipio VARCHAR, tipo VARCHAR (opcional)
Resultado	Lista de actores con conteo de servicios disponibles

[SQL]

```

-- QueryActoresPorMunicipio
SELECT
    a.id,
    a.nombre,
    a.tipo,
    a.descripcion,
    a.municipio,
    a.estado,
    a.telefono,
    a.correo,
    COUNT(s.id) AS total_servicios,
    COUNT(s.id) FILTER (WHERE s.es_gratuito = true) AS servicios_gratuitos
FROM
    actores a
    LEFT JOIN actores_servicios s ON s.actor_id = a.id AND s.activo = true

```

```

WHERE
    a.activo = true
    AND a.municipio ILIKE '%' || :municipio || '%'
    AND (:tipo IS NULL OR a.tipo = :tipo)
GROUP BY
    a.id, a.nombre, a.tipo, a.descripcion, a.municipio, a.estado, a.telefono, a.correo
ORDER BY
    a.nombre;

```

QueryServiciosPorDerecho

Atributo	Detalle
Nombre	QueryServiciosPorDerecho
Objetivo	Obtener todos los servicios disponibles que atienden un derecho específico
Parámetros	derecho_id INTEGER, municipio VARCHAR (opcional)
Resultado	Lista de servicios con datos del actor que los ofrece

[SQL]

```

-- QueryServiciosPorDerecho
SELECT
    s.id            AS servicio_id,
    s.nombre        AS servicio_nombre,
    s.descripcion   AS servicio_descripcion,
    s.tipo,
    s.es_gratuito,
    s.costo,
    s.disponibilidad,
    s.duracion_estimada,
    a.id            AS actor_id,
    a.nombre        AS actor_nombre,
    a.tipo          AS actor_tipo,
    a.municipio,
    a.telefono,
    a.correo,
    d.nombre        AS derecho_nombre,
    -- Horarios del actor
    (
        SELECT JSON_AGG(
            JSON_BUILD_OBJECT('dia', h.dia_semana, 'inicio', h.hora_inicio, 'fin', h.hora_fin)
        )
        FROM actores_horarios h
        WHERE h.actor_id = a.id AND h.activo = true
    ) AS horarios
FROM
    actores_servicios s
    JOIN actores a    ON a.id = s.actor_id
    JOIN derechos d    ON d.id = s.derecho_id
WHERE
    s.derecho_id = :derecho_id
    AND s.activo = true
    AND a.activo = true
    AND (:municipio IS NULL OR a.municipio ILIKE '%' || :municipio || '%')
ORDER BY
    a.nombre, s.nombre;

```

QueryPlanesPorNNA

Atributo	Detalle
Nombre	QueryPlanesPorNNA
Objetivo	Obtener todos los planes de restitución de un caso NNA con el porcentaje de avance calculado
Parámetros	caso_id INTEGER
Resultado	Lista de planes con avance promedio de medidas

[SQL]

```
-- QueryPlanesPorNNA
SELECT
  p.id,
  p.objetivo,
  p.estado,
  p.fecha_inicio,
  p.fecha_termino,
  p.observaciones,
  p.fecha_creacion,
  u.nombre      AS responsable_nombre,
  u.apellido_paterno,
  COUNT(m.id)    AS total_medidas,
  COUNT(m.id) FILTER (WHERE m.estado = 'completada') AS medidas_completadas,
  COALESCE(
    ROUND(AVG(m.porcentaje_avance)),
    0
  )              AS avance_promedio,
  (
    SELECT JSON_AGG(
      JSON_BUILD_OBJECT(
        'id', m2.id,
        'tipo', m2.tipo,
        'descripcion', m2.descripcion,
        'estado', m2.estado,
        'porcentaje_avance', m2.porcentaje_avance,
        'fecha_limite', m2.fecha_limite
      ) ORDER BY m2.fecha_creacion
    )
    FROM medidas_restitucion m2
    WHERE m2.plan_id = p.id
  )              AS medidas
FROM
  planes_restitucion p
  LEFT JOIN users u ON u.id = p.responsable_id
  LEFT JOIN medidas_restitucion m ON m.plan_id = p.id
WHERE
  p.caso_nna_id = :caso_id
GROUP BY
  p.id, p.objetivo, p.estado, p.fecha_inicio, p.fecha_termino,
  p.observaciones, p.fecha_creacion, u.nombre, u.apellido_paterno
ORDER BY
  p.fecha_creacion DESC;
```

QuerySeguimientosPlan

Atributo	Detalle
Nombre	QuerySeguimientosPlan
Objetivo	Obtener el historial de seguimientos de todas las medidas de un plan, ordenados cronológicamente
Parámetros	plan_id INTEGER
Resultado	Lista de seguimientos con datos del registrador y de la medida

[SQL]

```
-- QuerySeguimientosPlan
SELECT
  sm.id          AS seguimiento_id,
  sm.fecha_seguimiento,
  sm.descripcion_avance,
  sm.porcentaje_cumplimiento,
  sm.observaciones,
  sm.evidencias,
  sm.fecha_creacion,
  m.id          AS medida_id,
  m.tipo        AS medida_tipo,
  m.descripcion AS medida_descripcion,
  m.estado      AS medida_estado,
  m.porcentaje_avance AS medida_avance_actual,
  u.nombre      AS registrado_por_nombre,
  u.apellido_paterno AS registrado_por_apellido,
  u.rol         AS registrado_por_rol
FROM
  seguimientos_medida sm
  JOIN medidas_restitucion m ON m.id = sm.medida_id
  JOIN planes_restitucion p ON p.id = m.plan_id
  LEFT JOIN users u ON u.id = sm.registrado_por_id
WHERE
  p.id = :plan_id
ORDER BY
  sm.fecha_seguimiento DESC,
  sm.fecha_creacion DESC;
```

QueryFamiliograma

Atributo	Detalle
Nombre	QueryFamiliograma
Objetivo	Obtener el familiograma de un caso con todas las personas y relaciones para reconstruir el grafo
Parámetros	caso_id INTEGER
Resultado	Objeto con grafo_json, personas y relaciones

[SQL]

```
-- QueryFamiliograma – Paso 1: Canvas guardado
SELECT
  id,
  caso_id,
  grafo_json,
```



```

        version,
        fecha_actualizacion
FROM
    nna_familiogramas
WHERE
    caso_id = :caso_id
ORDER BY version DESC
LIMIT 1;

-- QueryFamiliograma – Paso 2: Personas del caso
SELECT
    id,
    nombre,
    edad,
    genero,
    rol_en_familia,
    tipo_simbolo,
    observaciones,
    vive_con_nna,
    es_responsable_legal
FROM
    nna_personas
WHERE
    caso_id = :caso_id
ORDER BY id;

-- QueryFamiliograma – Paso 3: Relaciones del caso
SELECT
    rf.id,
    rf.persona_origen_id,
    po.nombre AS persona_origen_nombre,
    rf.persona_destino_id,
    pd.nombre AS persona_destino_nombre,
    rf.tipo_relacion,
    rf.descripcion,
    rf.bidireccional
FROM
    nna_relaciones_familiares rf
    JOIN nna_personas po ON po.id = rf.persona_origen_id
    JOIN nna_personas pd ON pd.id = rf.persona_destino_id
WHERE
    rf.caso_id = :caso_id;

```

QueryUsuariosPorRol

Atributo	Detalle
Nombre	QueryUsuariosPorRol
Objetivo	Obtener la lista de usuarios activos con un rol específico (utilizado para asignar responsables en planes y medidas)
Parámetros	rol VARCHAR(30)
Resultado	Lista de usuarios con id y nombre completo

[SQL]

```

-- QueryUsuariosPorRol
SELECT
    u.id,

```

```

    u.nombre,
    u.apellido_paterno,
    u.apellido_materno,
    u.rol,
    u.correo,
    u.municipio,
    u.estado,
    CONCAT(u.nombre, ' ', u.apellido_paterno, ' ', u.apellido_materno) AS nombre_completo
FROM
    users u
WHERE
    u.activo = true
    AND (:rol IS NULL OR u.rol = :rol)
ORDER BY
    u.apellido_paterno, u.nombre;

```

5.4 Índices y Restricciones

Índices Definidos

Tabla	Columna(s)	Tipo	Justificación
users	correo	UNIQUE INDEX	Búsqueda de login
users	rfc	UNIQUE INDEX	Validación de unicidad
users	curp	UNIQUE INDEX	Validación de unicidad
nna_casos	creador_id	INDEX	Filtrado por creador
nna_casos	estado	INDEX	Filtrado por estado
diagnosticos	caso_nna_id	INDEX	Consulta de diagnósticos por caso
diagnosticos	responsable_id	INDEX	Consulta por responsable
derechos_vulnerados	diagnostico_id	INDEX	Consulta de derechos por diagnóstico
derechos_vulnerados	derecho_id	INDEX	Frecuencia por derecho
planes_restitucion	caso_nna_id	INDEX	Planes por caso
medidas_restitucion	plan_id	INDEX	Medidas por plan
seguimientos_medida	medida_id	INDEX	Seguimientos por medida
actores	municipio	INDEX	Búsqueda por municipio
actores_servicios	derecho_id	INDEX	Búsqueda por derecho
audit_logs	usuario_id	INDEX	Auditoría por usuario
audit_logs	fecha	INDEX	Auditoría por periodo

Restricciones de Integridad

Tabla	Restricción	Descripción
users	UNIQUE(correo)	No pueden existir dos usuarios con el mismo correo
users	CHECK(rol IN ('director','coordinador','psicologo','trabajador_social','legal','voluntario'))	Solo roles válidos
nna_casos	FK(creador_id) ON DELETE CASCADE	Al eliminar usuario, se eliminan sus casos
nna_tutores	FK(caso_id) ON DELETE CASCADE	Al eliminar caso, se elimina el tutor
diagnosticos	FK(caso_nna_id) ON DELETE CASCADE	Al eliminar caso, se eliminan sus diagnósticos
derechos_vulnerados	FK(diagnostico_id) ON DELETE CASCADE	Al eliminar diagnóstico, se eliminan sus derechos vulnerados
medidas_restitucion	CHECK(porcentaje_avance BETWEEN 0 AND 100)	Porcentaje en rango válido
seguimientos_medida	CHECK(porcentaje_cumplimiento BETWEEN 0 AND 100)	Porcentaje en rango válido
actores_servicios	FK(actor_id) ON DELETE CASCADE	Al eliminar actor, se eliminan sus servicios